

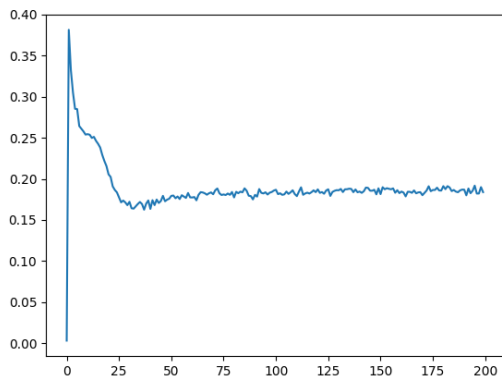
ECE 60146 HW 03

Andrea Goh (goh8@purdue.edu)

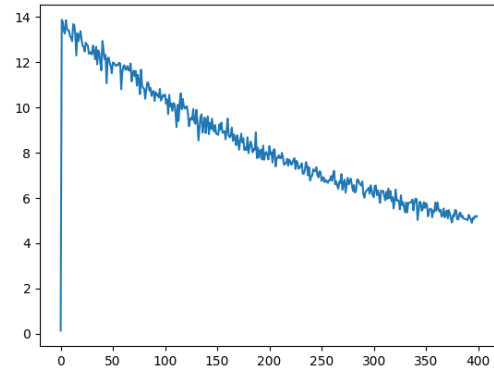
February 2025

1 Using ComputationalGraphPrimer

1.1 Comparing multi_neuron_classifier.py with torch.nn network



(a) **multi_neuron_classifier.py** network

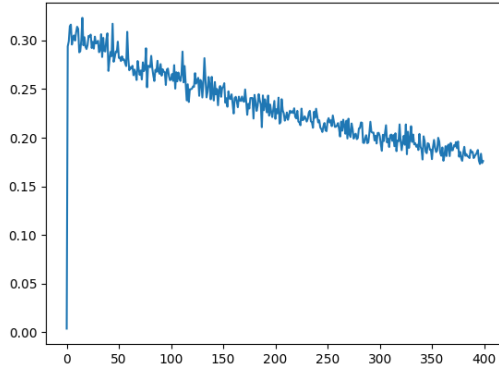


(b) **torch.nn** network

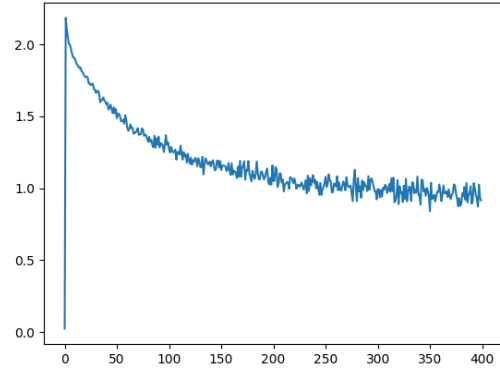
Figure 1: fig: Multi-neuron loss curve

Figure 1 shows the loss vs. iteration graph for the multi-neuron case. The handcrafted network seems to perform better as the loss starts out at a much smaller of approximately 0.37, compared to the initial loss of roughly 14 for the **torch.nn** network. The handcrafted network achieves the lowest loss of approximately 0.17 around iteration 25, at which the loss value flattens out, with a very slight increase. On the other hand, the loss from the **torch.nn** network steadily decreases to a value of 6 at iteration 400. Overall, we can say that the handcrafted network performed better as it achieves much lower loss values with a smaller number of iterations. However, loss decreases more stably in the **torch.nn** network.

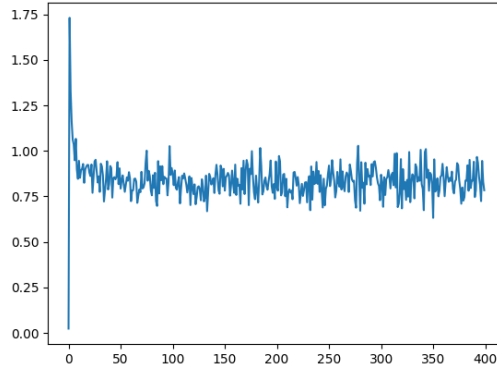
1.2 Comparing one_neuron_classifier.py with torch.nn network



(a) `one_neuron_classifier.py` network



(b) `torch.nn` network, `learning_rate = 1e-3`



(c) `torch.nn` network, `learning_rate = 5 * 1e-2`

Figure 2: fig: One-neuron loss curve

Figure 2 shows the loss vs. iteration graph for the one-neuron case. The handcrafted network outperformed the `torch.nn` network. At every iteration, the handcrafted network displays a lower loss, starting at 0.3 and steadily decreasing to approximately 0.19. On the other hand, the loss from the `torch.nn` network reached a minimum of around 0.8. As expected, when the `torch.nn` network had a higher learning rate, the minimum loss was obtained much quicker, at around 50 iterations, as compared to 400 iterations with the slower learning rate. The convergence is also more unstable with the higher learning rate.

1.3 SGD with Momentum (SGD+)

GD and SGD run into the same problem when the cost-function surface is a narrow valley, as opposed to a round valley. If the same learning rate is used for all iterations, after reaching the bottom of the valley, while the direction of movement is toward the optimum point, due to the learning rate, the solution path will crisscross the ravine. The problem with this is that we may never get a good sense of whether we are sufficiently close to the goal. One solution to this problem is to apply momentum in step-size calculations for learnable parameter values, and biases.

Let:

- w_t : model parameters at time step t
- $g_t = \nabla L(w_t)$: gradient of the loss function L at step t
- v_t : velocity (accumulated gradient)

- η : learning rate
- β : momentum coefficient

The basic update rule for SGD follows the equation: $w_{t+1} = w_t - \eta g_t$, where ηg_{t+1} represents the change in the solution path at time step t . Momentum involves carrying forward a portion of the previous update to influence the current step. These updates are computed as:

$$v_{t+1} = \beta v_t + g_t \quad (1)$$

$$w_{t+1} = w_t - \eta v_{t+1} \quad (2)$$

The way momentum works is that if the previous step is in the same direction as the current step, we go downhill quicker by increasing the step size. On the other hand, if the previous step is in the opposite direction as the current, we know that the optimum point was passed, and hence, the step size is reduced. This method helps navigate ravines more effectively by reducing oscillations and improving optimization stability.

1.3.1 Adaptive Moment Estimation (Adam)

Adam estimates on a running-average basis the first and second momentum, denoted as $\mathbf{m}_t$ and $\mathbf{v}_t$ respectively:

$$\mathbf{m}_t = \beta_1 * \mathbf{m}_{t-1} + (1 - \beta_1) * \text{grad}_t \quad (3)$$

$$\mathbf{v}_{t+1} = \beta_2 * \mathbf{v}_t + (1 - \beta_2) * (\text{grad}_t)^2 \quad (4)$$

Typically, the constant values for β_1 and β_2 are close to 1. The recursion is also initialized such that $m_0 = v_0 = 0$. However, this zero initialization results in a bias towards zero for the two momentum for several training iterations. To address this flaw, bias-corrected moments $\hat{m}$ and $\hat{v}$ can be used for the first and second moments, which can be found using the following equations:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (5)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (6)$$

Subsequently, we can find the update parameter vector using the following equation:

$$p_{t+1} = p_t - lr * \frac{\hat{m}_{t+1}}{\sqrt{\hat{v}_{t+1} + \epsilon}} \quad (7)$$

2 Programming Task

2.1 One-Neuron Models

2.1.1 Description

For the one-neuron models, I created three subclasses that inherit from **ComputationalGraphPrimer**. The functions within each of these subclasses were taken directly from: <https://engineering.purdue.edu/kak/distCGP/>, and comments are written wherever I made modifications.

The first subclass is called **OneNeuronSGD**, which is for the vanilla stochastic gradient descent. The `__init__` method of this class uses the `super()` function to inherit from the `ComputationalGraphPrimer` class. The `run_training_loop_one_neuron_model` was modified by removing the plotting functions for loss and instead returning `loss_running_record`.

The second subclass is called **OneNeuronSGDPlus**, which is for SGD+. The `__init__` method has an additional parameter called `beta`, which is the momentum coefficient. In addition, instance variables for velocity weights and biases are initialized to 0 in the `run_training_loop_one_neuron_model`. These variables are used in the `backprop_and_update_params_one_neuron_model` function to apply momentum to the weight and bias updates.

The third subclass is called **OneNeuronSGDAdam**. The `__init__` method includes three additional parameters, `beta_1` (first moment decay rate), `beta_2` (second moment decay rate), and `epsilon`. Instance variables for the first and second moment are initialized for both weights and biases in the `run_training_loop_one_neuron_model` function. These variables are used in the `backprop_and_update_params_one_neuron_model` to update the weights and moments using the Adam update equations.

2.1.2 Code

```

1 class OneNeuronSGD(ComputationalGraphPrimer):
2     ##### Modification ----- Start ----- #####
3     # The OneNeuronSGD is created to be a subclass of the ComputationalGraphPrimer class
4     # super() is used to inherit functions from the CGP class
5     def __init__(self, *args, ** kwargs):
6         super().__init__(*args, ** kwargs) # Inherit from CGP class
7     ##### Modification ----- End ----- #####
8
9     def run_training_loop_one_neuron_model(self, training_data):
10        self.vals_for_learnable_params = {param: random.uniform(0,1) for param in self.
11            learnable_params}
12        self.bias = random.uniform(0,1)
13        class DataLoader:
14            def __init__(self, training_data, batch_size):
15                self.training_data = training_data
16                self.batch_size = batch_size
17                self.class_0_samples = [(item, 0) for item in self.training_data[0]]
18                self.class_1_samples = [(item, 1) for item in self.training_data[1]]
19            def __len__(self):
20                return len(self.training_data[0]) + len(self.training_data[1])
21            def _getitem(self):
22                cointoss = random.choice([0,1])
23                if cointoss == 0:
24                    return random.choice(self.class_0_samples)
25                else:
26                    return random.choice(self.class_1_samples)
27            def getbatch(self):
28                batch_data, batch_labels = [], []
29                maxval = 0.0
30                for _ in range(self.batch_size):
31                    item = self._getitem()
32                    if np.max(item[0]) > maxval:
33                        maxval = np.max(item[0])
34                    batch_data.append(item[0])
35                    batch_labels.append(item[1])
36                batch_data = [item/maxval for item in batch_data]
37                batch = [batch_data, batch_labels]
38                return batch
39        data_loader = DataLoader(training_data, batch_size=self.batch_size)
40        loss_running_record = []
41        i = 0
42        avg_loss_over_iterations = 0.0
43        for i in range(self.training_iterations):
44            data = data_loader.getbatch()
45            data_tuples_in_batch = data[0]
46            class_labels_in_batch = data[1]
47            y_preds, deriv_sigmoids = self.forward_prop_one_neuron_model(
48                data_tuples_in_batch)
49            loss = sum([(abs(class_labels_in_batch[i] - y_preds[i]))**2 for i in range(
50                len(class_labels_in_batch))])
51            avg_loss_over_iterations += loss / float(len(class_labels_in_batch))
52            if i%(self.display_loss_how_often) == 0:
53                avg_loss_over_iterations /= self.display_loss_how_often
54                loss_running_record.append(avg_loss_over_iterations)
55                print("[iter=%d] loss=%.4f" % (i+1, avg_loss_over_iterations))
56                avg_loss_over_iterations = 0.0
57            y_errors_in_batch = list(map(operator.sub, class_labels_in_batch, y_preds))
58            self.backprop_and_update_params_one_neuron_model(data_tuples_in_batch,
59                y_preds, y_errors_in_batch, deriv_sigmoids)
60            # Remove plots
61            return loss_running_record # return loss record
62
63 class OneNeuronSGDPlus(ComputationalGraphPrimer):
64     ##### Modification ----- Start ----- #####
65     # The OneNeuronSGDPlus is created to be a subclass of the ComputationalGraphPrimer
66     class
67     # super() is used to inherit functions from the CGP class
68     def __init__(self, beta, *args, ** kwargs): # Add beta to __init__ parameter
69         super().__init__(*args, ** kwargs) # Inherit from CGP class
70         self.beta = beta # Initialize Momentum coefficient
71     ##### Modification ----- End ----- #####

```

```

67
68 def run_training_loop_one_neuron_model(self, training_data):
69     self.vals_for_learnable_params = {param: random.uniform(0,1) for param in self.
70         learnable_params}
71     self.bias = random.uniform(0,1)
72
73     ##### Modification ----- Start ----- #####
74     # Initialize velocity (accumulated gradient) for weights and biases
75     # Velocity will be used for momentum update calculations
76     self.v_weights = {param: 0.0 for param in self.learnable_params} # Set all
77     weights to 0
78     self.v_bias = 0 # Initialize bias as 0
79     ##### Modification ----- End ----- #####
80
81     class DataLoader:
82     def __init__(self, training_data, batch_size):
83         self.training_data = training_data
84         self.batch_size = batch_size
85         self.class_0_samples = [(item, 0) for item in self.training_data[0]]
86         self.class_1_samples = [(item, 1) for item in self.training_data[1]]
87     def __len__(self):
88         return len(self.training_data[0]) + len(self.training_data[1])
89     def _getitem(self):
90         cointoss = random.choice([0,1])
91         if cointoss == 0:
92             return random.choice(self.class_0_samples)
93         else:
94             return random.choice(self.class_1_samples)
95     def getbatch(self):
96         batch_data, batch_labels = [], []
97         maxval = 0.0
98         for _ in range(self.batch_size):
99             item = self._getitem()
100             if np.max(item[0]) > maxval:
101                 maxval = np.max(item[0])
102             batch_data.append(item[0])
103             batch_labels.append(item[1])
104         batch_data = [item/maxval for item in batch_data]
105         batch = [batch_data, batch_labels]
106         return batch
107
108     data_loader = DataLoader(training_data, batch_size=self.batch_size)
109     loss_running_record = []
110     i = 0
111     avg_loss_over_iterations = 0.0
112     for i in range(self.training_iterations):
113         data = data_loader.getbatch()
114         data_tuples_in_batch = data[0]
115         class_labels_in_batch = data[1]
116         y_preds, deriv_sigmoids = self.forward_prop_one_neuron_model(
117             data_tuples_in_batch)
118         loss = sum([(abs(class_labels_in_batch[i] - y_preds[i]))**2 for i in range(
119             len(class_labels_in_batch))])
120         avg_loss_over_iterations += loss / float(len(class_labels_in_batch))
121         if i%(self.display_loss_how_often) == 0:
122             avg_loss_over_iterations /= self.display_loss_how_often
123             loss_running_record.append(avg_loss_over_iterations)
124             print("[iter=%d] loss=%.4f" % (i+1, avg_loss_over_iterations))
125             avg_loss_over_iterations = 0.0
126         y_errors_in_batch = list(map(operator.sub, class_labels_in_batch, y_preds))
127         self.backprop_and_update_params_one_neuron_model(data_tuples_in_batch,
128             y_preds, y_errors_in_batch, deriv_sigmoids)
129
130     # Remove plots
131     return loss_running_record # return loss record
132
133 def backprop_and_update_params_one_neuron_model(self, data_tuples_in_batch,
134     predictions, y_errors_in_batch, deriv_sigmoids):
135     input_vars = self.independent_vars
136     input_vars_to_param_map = self.var_to_var_param[self.output_vars[0]]
137     param_to_vars_map = {param : var for var, param in input_vars_to_param_map.items
138         ()}
139     vals_for_learnable_params = self.vals_for_learnable_params
140     for i,param in enumerate(self.vals_for_learnable_params):
141         partial_of_loss_wrt_param = 0.0

```

```

133         for j in range(self.batch_size):
134             vals_for_input_vars_dict = dict(zip(input_vars, list(
135                 data_tuples_in_batch[j])))
136             partial_of_loss_wrt_param += - y_errors_in_batch[j] *
137                 vals_for_input_vars_dict[param_to_vars_map[param]] * deriv_sigmoids[
138                 j]
139             partial_of_loss_wrt_param /= float(self.batch_size)
140
141             ##### Modification ----- Start ----- #####
142             # This modification updates the learnable parameters using momentum
143             self.v_weights[param] = self.beta * self.v_weights[param] +
144                 partial_of_loss_wrt_param # Update velocity (accumulated gradient)
145             step = -self.learning_rate * self.v_weights[param] # Step size = learning
146                 rate * velocity
147             ##### Modification ----- End ----- #####
148
149             # step = self.learning_rate * partial_of_loss_wrt_param
150             self.vals_for_learnable_params[param] += step
151             y_error_avg = sum(y_errors_in_batch) / float(self.batch_size)
152             deriv_sigmoid_avg = sum(deriv_sigmoids) / float(self.batch_size)
153
154             ##### Modification ----- Start ----- #####
155             # This modification updates the biases using momentum
156             self.v_bias = self.beta * self.v_bias + y_error_avg * deriv_sigmoid_avg
157             self.bias += self.learning_rate * self.v_bias # Update bias
158             ##### Modification ----- End ----- #####
159             # self.bias += self.learning_rate * y_error_avg * deriv_sigmoid_avg
160
161     class OneNeuronSGDAdam(ComputationalGraphPrimer):
162         ##### Modification ----- Start ----- #####
163         # The OneNeuronSGDPlus is created to be a subclass of the ComputationalGraphPrimer
164         class
165         # super() is used to inherit functions from the CGP class
166         def __init__(self, beta_1, beta_2, epsilon, *args, ** kwargs):
167             super().__init__(*args, ** kwargs) # Inherit from CGP class
168             self.beta_1 = beta_1 # Initialize decay rate for first moment
169             self.beta_2 = beta_2 # Initialize decay rate for second moment
170             self.epsilon = epsilon
171             ##### Modification ----- End ----- #####
172
173         def run_training_loop_one_neuron_model(self, training_data):
174             self.vals_for_learnable_params = {param: random.uniform(0,1) for param in self.
175                 learnable_params}
176             self.bias = random.uniform(0,1)
177
178             ##### Modification ----- Start ----- #####
179             # Initialize first moment (m) and second moment (v) for both weights and bias
180             self.v_weights = {param: 0.0 for param in self.learnable_params}
181             self.v_bias = 0
182             self.m_weights = {param: 0.0 for param in self.learnable_params}
183             self.m_bias = 0
184             self.timestep = 1 # Initialize time step for calculation of bias corected values
185             ##### Modification ----- End ----- #####
186
187         class DataLoader:
188             def __init__(self, training_data, batch_size):
189                 self.training_data = training_data
190                 self.batch_size = batch_size
191                 self.class_0_samples = [(item, 0) for item in self.training_data[0]]
192                 self.class_1_samples = [(item, 1) for item in self.training_data[1]]
193             def __len__(self):
194                 return len(self.training_data[0]) + len(self.training_data[1])
195             def _getitem(self):
196                 Programming
197                 cointoss = random.choice([0,1])
198                 if cointoss == 0:
199                     return random.choice(self.class_0_samples)
200                 else:
201                     return random.choice(self.class_1_samples)
202             def getbatch(self):
203                 batch_data, batch_labels = [], []
204                 maxval = 0.0
205                 for _ in range(self.batch_size):
206                     item = self._getitem()

```

```

199         if np.max(item[0]) > maxval:
200             maxval = np.max(item[0])
201             batch_data.append(item[0])
202             batch_labels.append(item[1])
203         batch_data = [item/maxval for item in batch_data]
204         batch = [batch_data, batch_labels]
205         return batch
206     data_loader = DataLoader(training_data, batch_size=self.batch_size)
207     loss_running_record = []
208     i = 0
209     avg_loss_over_iterations = 0.0
210     for i in range(self.training_iterations):
211         data = data_loader.getbatch()
212         data_tuples_in_batch = data[0]
213         class_labels_in_batch = data[1]
214         y_preds, deriv_sigmoids = self.forward_prop_one_neuron_model(
215             data_tuples_in_batch)
216         loss = sum([(abs(class_labels_in_batch[i] - y_preds[i]))**2 for i in range(
217             len(class_labels_in_batch))])
218         avg_loss_over_iterations += loss / float(len(class_labels_in_batch))
219         if i%(self.display_loss_how_often) == 0:
220             avg_loss_over_iterations /= self.display_loss_how_often
221             loss_running_record.append(avg_loss_over_iterations)
222             print("[iter=%d] loss=%.4f" % (i+1, avg_loss_over_iterations))
223             avg_loss_over_iterations = 0.0
224             y_errors_in_batch = list(map(operator.sub, class_labels_in_batch, y_preds))
225             self.backprop_and_update_params_one_neuron_model(data_tuples_in_batch,
226                 y_preds, y_errors_in_batch, deriv_sigmoids)
227     # Remove plots
228     return loss_running_record # return loss record
229
230 def backprop_and_update_params_one_neuron_model(self, data_tuples_in_batch,
231     predictions, y_errors_in_batch, deriv_sigmoids):
232     input_vars = self.independent_vars
233     input_vars_to_param_map = self.var_to_var_param[self.output_vars[0]]
234     param_to_vars_map = {param : var for var, param in input_vars_to_param_map.items
235         ()}
236     vals_for_learnable_params = self.vals_for_learnable_params
237     for i,param in enumerate(self.vals_for_learnable_params):
238         partial_of_loss_wrt_param = 0.0
239         for j in range(self.batch_size):
240             vals_for_input_vars_dict = dict(zip(input_vars, list(
241                 data_tuples_in_batch[j])))
242             partial_of_loss_wrt_param += - y_errors_in_batch[j] *
243                 vals_for_input_vars_dict[param_to_vars_map[param]] * deriv_sigmoids[
244                     j]
245             partial_of_loss_wrt_param /= float(self.batch_size)
246             ##### Modification ----- Start ----- #####
247             # This modification updates weighs using fist and second moments
248             self.m_weights[param] = self.beta_1 * self.m_weights[param] + (1 - self.
249                 beta_1) * partial_of_loss_wrt_param # First moment
250             self.v_weights[param] = self.beta_2 * self.v_weights[param] + (1 - self.
251                 beta_2) * partial_of_loss_wrt_param**2 # Second moment
252             m_hat = self.m_weights[param] / (1 - self.beta_1**self.timestep) # Bias
253                 corrected value for first moment
254             v_hat = self.v_weights[param] / (1 - self.beta_2**self.timestep) # Bias
255                 corrected value for second moment
256             step = -self.learning_rate * m_hat / np.sqrt(v_hat + self.epsilon)
257             ##### Modification ----- End ----- #####
258
259             # step = self.learning_rate * partial_of_loss_wrt_param
260             self.vals_for_learnable_params[param] += step
261     y_error_avg = sum(y_errors_in_batch) / float(self.batch_size)
262     deriv_sigmoid_avg = sum(deriv_sigmoids) / float(self.batch_size)
263
264     ##### Modification ----- Start ----- #####
265     # This modification updates bias using first and second moments
266     self.m_bias = self.beta_1 * self.m_bias + (1 - self.beta_1) * y_error_avg *
267         deriv_sigmoid_avg
268     self.v_bias = self.beta_2 * self.v_bias + (1 - self.beta_2) * (y_error_avg *
269         deriv_sigmoid_avg)**2 # Update velocity
270     m_hat = self.m_bias / (1 - self.beta_1**self.timestep) # Bias corrected value
271         for first moment

```

```

257     v_hat = self.v_bias / (1 - self.beta_2**self.timestep) # Bias corrected value
        for second moment
258     self.bias += self.learning_rate * m_hat / (np.sqrt(v_hat + self.epsilon)) #
        Update bias
259     self.timestep += 1 # Increment time step
260     ##### Modification ----- End ----- #####
261
262     # self.bias += self.learning_rate * y_error_avg * deriv_sigmoid_avg

```

2.1.3 Performance Analysis

From figure 3, we can see that for all three learning rates, both SGD+ and Adam significantly outperformed the vanilla SGD. When the learning rate is 0.01, the loss from SGD decreases up until iteration 70 and diverges with subsequent iterations. Thus, with a learning rate of 0.01, SGD fails. On the other hand, both SGD+ and Adam converges to a similar final loss value. Adam converges faster, while SGD+ seems to be more stable as its loss curve appears smoother than that of Adam.

When the learning rate is 0.001, SGD converges with a much higher loss value compared to SGD+ and Adam. This could be due to the fact that SGD converged to some local minimum, while SGD+ and Adam converged to the global minimum. Adam achieves a lower final loss value and also converges faster than SGD+. However, SGD+ seems to have a more stable convergence behavior.

Finally, at the learning rate of 0.0001, the loss for SGD does not converge but instead oscillates between a constant value of approximately 0.3. Both SGD+ and Adam converge. Initially the loss for Adam is larger than SGD+ but the final loss for Adam is slightly lower than SGD+. Thus, the rate of convergence for Adam is again faster than SGD+, while SGD+ exhibits more stable convergence behavior. For both SGD+ and Adam, it seems that a learning rate of 0.0001 is too small because both curves do not flatten out by 400 iterations. Thus, we may expect the loss to decrease even further with more iterations.

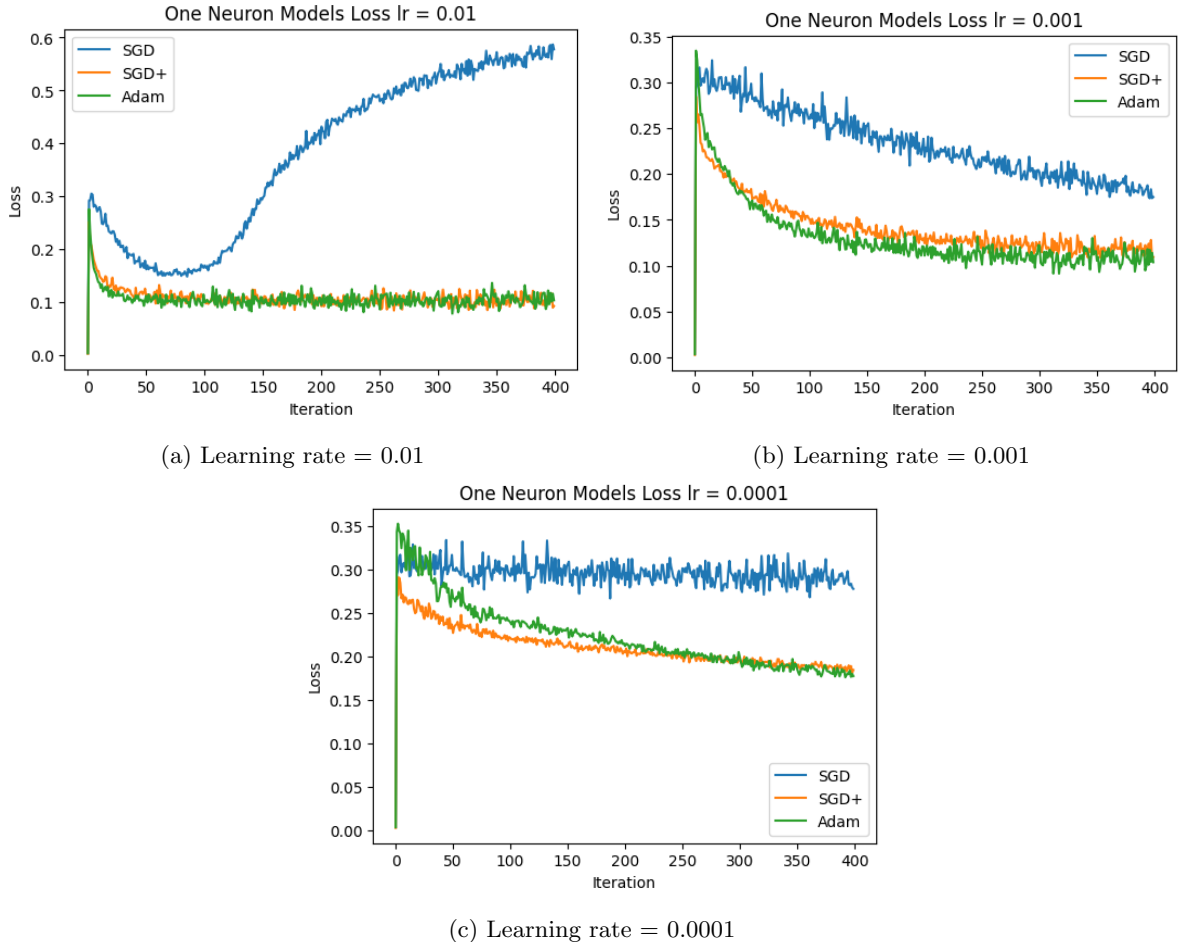


Figure 3: fig: One-neuron model comparisons

2.1.4 Adam Optimizer Hyperparameters

In this section, we explore how the Adam optimizer's performance is affected by its hyperparameters β_1 and β_2 . From table 1, we can see that the performance of the Adam optimizer is optimized when $\beta_1 = 0.95$ and $\beta_2 = 0.9$. The time taken for each configuration are very similar, and is not impacted by the performance.

Table 1: Comparison of Adam Hyperparameters

beta_1	beta_2	Time Taken	Final Loss	Minimum Loss
0.8	0.89	7.66711	0.109532	0.0925391
0.8	0.9	7.46425	0.0895824	0.0866681
0.8	0.95	7.43318	0.113911	0.0836033
0.95	0.89	7.34527	0.0966654	0.0888071
0.95	0.9	7.25438	0.0859667	0.0830452
0.95	0.95	7.48425	0.0940357	0.0885052
0.99	0.89	6.96183	0.106588	0.0841122
0.99	0.9	7.11673	0.0966726	0.083119
0.99	0.95	7.02156	0.109321	0.0778785

2.2 Multi Neuron Models

2.2.1 Description

For the multi-neuron models, I created three subclasses that inherit from **ComputationalGraphPrimer**. Again, the functions within each of these subclasses were taken directly from: <https://engineering.purdue.edu/kak/distCGP/>, and comments are written wherever I made modifications. The three subclasses are called **MultiNeuronSGD**, **MultiNeuronSGDPlus**, and **MultiNeuronSGDAdam**, corresponding to vanilla SGD, SGD+, and SGD with Adam optimizer respectively. The modifications made to the code are similar to the modifications made in the one-neuron case.

2.2.2 Code

```
1 class MultiNeuronSGD(ComputationalGraphPrimer):
2     ##### Modification ----- Start ----- #####
3     # The MultiNeuronSGD is created to be a subclass of the ComputationalGraphPrimer
4     class
5     # super() is used to inherit functions from the CGP class
6     def __init__(self, *args, ** kwargs):
7         super().__init__(*args, ** kwargs) # Inherit from CGP class
8     ##### Modification ----- End ----- #####
9
10    def run_training_loop_multi_neuron_model(self, training_data):
11        class DataLoader:
12            def __init__(self, training_data, batch_size):
13                self.training_data = training_data
14                self.batch_size = batch_size
15                self.class_0_samples = [(item, 0) for item in self.training_data[0]]
16                self.class_1_samples = [(item, 1) for item in self.training_data[1]]
17            def __len__(self):
18                return len(self.training_data[0]) + len(self.training_data[1])
19            def _getitem(self):
20                cointoss = random.choice([0,1])
21                if cointoss == 0:
22                    return random.choice(self.class_0_samples)
23                else:
24                    return random.choice(self.class_1_samples)
25            def getbatch(self):
26                batch_data, batch_labels = [], []
27                maxval = 0.0
28                for _ in range(self.batch_size):
29                    item = self._getitem()
30                    if np.max(item[0]) > maxval:
31                        maxval = np.max(item[0])
32                batch_data.append(item[0])
```

```

32         batch_labels.append(item[1])
33         batch_data = [item/maxval for item in batch_data]
34         batch = [batch_data, batch_labels]
35         return batch
36     self.vals_for_learnable_params = {param: random.uniform(0,1) for param in self.
37         learnable_params}
38     self.bias = {i : [random.uniform(0,1) for j in range( self.layers_config[i] )
39         ] for i in range(1, self.num_layers)}
40     data_loader = DataLoader(training_data, batch_size=self.batch_size)
41     loss_running_record = []
42     i = 0
43     avg_loss_over_iterations = 0.0
44     for i in range(self.training_iterations):
45         data = data_loader.getbatch()
46         data_tuples = data[0]
47         class_labels = data[1]
48         self.forward_prop_multi_neuron_model(data_tuples)
49         predicted_labels_for_batch = self.forw_prop_vals_at_layers[self.num_layers
50             -1]
51         y_preds = [item for sublist in predicted_labels_for_batch for item in
52             sublist]
53         loss = sum([(abs(class_labels[i] - y_preds[i]))**2 for i in range(len(
54             class_labels))])
55         loss_avg = loss / float(len(class_labels))
56         avg_loss_over_iterations += loss_avg
57         if i%(self.display_loss_how_often) == 0:
58             avg_loss_over_iterations /= self.display_loss_how_often
59             loss_running_record.append(avg_loss_over_iterations)
60             print("[iter=%d] loss=%.4f" % (i+1, avg_loss_over_iterations))
61             avg_loss_over_iterations = 0.0
62         y_errors_in_batch = list(map(operator.sub, class_labels, y_preds))
63         self.backprop_and_update_params_multi_neuron_model(y_preds,
64             y_errors_in_batch)
65     # Remove plots
66     return loss_running_record
67
68 class MultiNeuronSGDPlus(ComputationalGraphPrimer):
69     ##### Modification ----- Start ----- #####
70     # The MultiNeuronSGDPlus is created to be a subclass of the ComputationalGraphPrimer
71     class
72     # super() is used to inherit functions from the CGP class
73     def __init__(self, beta, *args, ** kwargs):
74         super().__init__(*args, ** kwargs) # Inherit from CGP class
75         self.beta = beta # Initialize Momentum coefficient
76     ##### Modification ----- End ----- #####
77
78     def run_training_loop_multi_neuron_model(self, training_data):
79         class DataLoader:
80             def __init__(self, training_data, batch_size):
81                 self.training_data = training_data
82                 self.batch_size = batch_size
83                 self.class_0_samples = [(item, 0) for item in self.training_data[0]]
84                 self.class_1_samples = [(item, 1) for item in self.training_data[1]]
85             def __len__(self):
86                 return len(self.training_data[0]) + len(self.training_data[1])
87             def _getitem(self):
88                 cointoss = random.choice([0,1])
89                 if cointoss == 0:
90                     return random.choice(self.class_0_samples)
91                 else:
92                     return random.choice(self.class_1_samples)
93             def getbatch(self):
94                 batch_data, batch_labels = [], []
95                 maxval = 0.0
96                 for _ in range(self.batch_size):
97                     item = self._getitem()
98                     if np.max(item[0]) > maxval:
99                         maxval = np.max(item[0])
100                     batch_data.append(item[0])
101                     batch_labels.append(item[1])
102                 batch_data = [item/maxval for item in batch_data]
103                 batch = [batch_data, batch_labels]
104                 return batch

```

```

98     self.vals_for_learnable_params = {param: random.uniform(0,1) for param in self.
    learnable_params}
99     self.bias =    {i : [random.uniform(0,1) for j in range( self.layers_config[i] )
    ] for i in range(1, self.num_layers)}
100
101     ##### Modification ----- Start ----- #####
102     # Initialize velocity (accumulated gradient) for weights and biases
103     # Velocity will be used for momentum update calculations
104     self.v_weights = {param: 0.0 for param in self.learnable_params} # Set all
    weights to 0
105     self.v_bias = {i: [0.0 for _ in range(self.layers_config[i])] for i in range(1,
    self.num_layers)} # Initialize bias as 0
106     ##### Modification ----- End ----- #####
107
108     data_loader = DataLoader(training_data, batch_size=self.batch_size)
109     loss_running_record = []
110     i = 0
111     avg_loss_over_iterations = 0.0
112     for i in range(self.training_iterations):
113         data = data_loader.getbatch()
114         data_tuples = data[0]
115         class_labels = data[1]
116         self.forward_prop_multi_neuron_model(data_tuples)
117         predicted_labels_for_batch = self.forw_prop_vals_at_layers[self.num_layers
    -1]
118         y_preds = [item for sublist in predicted_labels_for_batch for item in
    sublist]
119         loss = sum([(abs(class_labels[i] - y_preds[i]))**2 for i in range(len(
    class_labels))])
120         loss_avg = loss / float(len(class_labels))
121         avg_loss_over_iterations += loss_avg
122         if i%(self.display_loss_how_often) == 0:
123             avg_loss_over_iterations /= self.display_loss_how_often
124             loss_running_record.append(avg_loss_over_iterations)
125             print("[iter=%d] loss=%.4f" % (i+1, avg_loss_over_iterations))
126             avg_loss_over_iterations = 0.0
127             y_errors_in_batch = list(map(operator.sub, class_labels, y_preds))
128             self.backprop_and_update_params_multi_neuron_model(y_preds,
    y_errors_in_batch)
129         # Remove plots
130         return loss_running_record
131
132     def backprop_and_update_params_multi_neuron_model(self, predictions, y_errors):
133         pred_err_backproped_at_layers =    [{i : [None for j in range( self.
    layers_config[i] ) ] for i in range(self.num_layers)} for _ in range(self.
    batch_size) ]
134         partial_of_loss_wrt_params = {param : 0.0 for param in self.all_params}
135         bias_changes =    {i : [0.0 for j in range( self.layers_config[i] ) ] for i in
    range(1, self.num_layers)}
136         for b in range(self.batch_size):
137             pred_err_backproped_at_layers[b][self.num_layers - 1] = [ y_errors[b] ]
138             for back_layer_index in reversed(range(1,self.num_layers)):
139                 input_vals = self.forw_prop_vals_at_layers[back_layer_index -1]
140                 deriv_sigmoids = self.gradient_vals_for_layers[back_layer_index]
141                 vars_in_layer = self.layer_vars[back_layer_index]
142                 vars_in_next_layer_back = self.layer_vars[back_layer_index - 1]
143                 vals_for_input_vars_dict = dict(zip(vars_in_next_layer_back, self.
    forw_prop_vals_at_layers[back_layer_index - 1][b]))
144                 layer_params = self.layer_params[back_layer_index]
145                 transposed_layer_params = list(zip(*layer_params))
146                 for k,var1 in enumerate(vars_in_next_layer_back):
147                     for j,var2 in enumerate(vars_in_layer):
148                         pred_err_backproped_at_layers[b][back_layer_index - 1][k] = sum
    ([self.vals_for_learnable_params[transposed_layer_params[k][
    i]] * pred_err_backproped_at_layers[b][back_layer_index][i]
    for i in range(len(vars_in_layer))])
149                 for j,var in enumerate(vars_in_layer):
150                     layer_params = self.layer_params[back_layer_index][j]
151                     input_vars_to_param_map = self.var_to_var_param[var]
152                     param_to_vars_map = {param : var for var, param in
    input_vars_to_param_map.items()}
153                     for i,param in enumerate(layer_params):

```

```

154         partial_of_loss_wrt_params[param] +=
            pred_err_backproped_at_layers[b][back_layer_index][j] *
            vals_for_input_vars_dict[param_to_vars_map[param]] *
            deriv_sigmoids[b][j]
155     for k,var1 in enumerate(vars_in_next_layer_back):
156         for j,var2 in enumerate(vars_in_layer):
157             if back_layer_index-1 > 0:
158                 bias_changes[back_layer_index-1][k] +=
                    pred_err_backproped_at_layers[b][back_layer_index - 1][k]
                    * deriv_sigmoids[b][j]
159     for param in partial_of_loss_wrt_params:
160         partial_of_loss_wrt_param = partial_of_loss_wrt_params[param] / float(self.
            batch_size)
161
162     ##### Modification ----- Start ----- #####
163     self.v_weights[param] = self.beta * self.v_weights[param] +
        partial_of_loss_wrt_param
164     step = self.learning_rate * self.v_weights[param]
165     ##### Modification ----- End ----- #####
166
167     # step = self.learning_rate * partial_of_loss_wrt_param
168     self.vals_for_learnable_params[param] += step
169     for layer_index in range(1,self.num_layers):
170         for k in range(self.layers_config[layer_index]):
171             ##### Modification ----- Start ----- #####
172             self.v_bias[layer_index][k] = self.beta * self.v_bias[layer_index][k] +
                ( bias_changes[layer_index][k] / float(self.batch_size) ) # Update
                momentum
173             self.bias[layer_index][k] += self.learning_rate * self.v_bias[
                layer_index][k] # Update Bias
174             ##### Modification ----- End ----- #####
175
176             # self.bias[layer_index][k] += self.learning_rate * ( bias_changes[
                layer_index][k] / float(self.batch_size) )
177
178 class MultiNeuronSGDAdam(ComputationalGraphPrimer):
179     ##### Modification ----- Start ----- #####
180     # The MultiNeuronSGDAdam is created to be a subclass of the ComputationalGraphPrimer
        class
181     # super() is used to inherit functions from the CGP class
182     def __init__(self, beta_1, beta_2, epsilon, *args, ** kwargs):
183         super().__init__(*args, ** kwargs) # Inherit from CGP class
184         self.beta_1 = beta_1 # Initialize decay rate for first moment
185         self.beta_2 = beta_2 # Initialize decay rate for second moment
186         self.epsilon = epsilon
187     ##### Modification ----- End ----- #####
188
189     def run_training_loop_multi_neuron_model(self, training_data):
190         class DataLoader:
191             def __init__(self, training_data, batch_size):
192                 self.training_data = training_data
193                 self.batch_size = batch_size
194                 self.class_0_samples = [(item, 0) for item in self.training_data[0]]
195                 self.class_1_samples = [(item, 1) for item in self.training_data[1]]
196             def __len__(self):
197                 return len(self.training_data[0]) + len(self.training_data[1])
198             def _getitem(self):
199                 cointoss = random.choice([0,1])
200                 if cointoss == 0:
201                     return random.choice(self.class_0_samples)
202                 else:
203                     return random.choice(self.class_1_samples)
204             def getbatch(self):
205                 batch_data, batch_labels = [],[]
206                 maxval = 0.0
207                 for _ in range(self.batch_size):
208                     item = self._getitem()
209                     if np.max(item[0]) > maxval:
210                         maxval = np.max(item[0])
211                     batch_data.append(item[0])
212                     batch_labels.append(item[1])
213                 batch_data = [item/maxval for item in batch_data]
214                 batch = [batch_data, batch_labels]

```

```

215         return batch
216     self.vals_for_learnable_params = {param: random.uniform(0,1) for param in self.
        learnable_params}
217     self.bias = {i : [random.uniform(0,1) for j in range( self.layers_config[i] )
        ] for i in range(1, self.num_layers)}
218
219     ##### Modification ----- Start ----- #####
220     # Initialize first moment (m) and second moment (v) for both weights and bias
221     self.v_weights = {param: 0.0 for param in self.learnable_params}
222     self.v_bias = {i: [0.0 for _ in range(self.layers_config[i])] for i in range(1,
        self.num_layers)}
223     self.m_weights = {param: 0.0 for param in self.learnable_params}
224     self.m_bias = {i: [0.0 for _ in range(self.layers_config[i])] for i in range(1,
        self.num_layers)}
225     self.timestep = 1 # Initialize time step for calculation of bias corected values
226     ##### Modification ----- End ----- #####
227
228     data_loader = DataLoader(training_data, batch_size=self.batch_size)
229     loss_running_record = []
230     i = 0
231     avg_loss_over_iterations = 0.0
232     for i in range(self.training_itations):
233         data = data_loader.getbatch()
234         data_tuples = data[0]
235         class_labels = data[1]
236         self.forward_prop_multi_neuron_model(data_tuples)
237         predicted_labels_for_batch = self.forw_prop_vals_at_layers[self.num_layers
            -1]
238         y_preds = [item for sublist in predicted_labels_for_batch for item in
            sublist]
239         loss = sum([(abs(class_labels[i] - y_preds[i]))**2 for i in range(len(
            class_labels))])
240         loss_avg = loss / float(len(class_labels))
241         avg_loss_over_iterations += loss_avg
242         if i%(self.display_loss_how_often) == 0:
243             avg_loss_over_iterations /= self.display_loss_how_often
244             loss_running_record.append(avg_loss_over_iterations)
245             print("[iter=%d] loss=%.4f" % (i+1, avg_loss_over_iterations))
246             avg_loss_over_iterations = 0.0
247             y_errors_in_batch = list(map(operator.sub, class_labels, y_preds))
248             self.backprop_and_update_params_multi_neuron_model(y_preds,
                y_errors_in_batch)
249         # Remove plots
250     return loss_running_record
251
252     def backprop_and_update_params_multi_neuron_model(self, predictions, y_errors):
253         pred_err_backproped_at_layers = [ {i : [None for j in range( self.
            layers_config[i] ) ] for i in range(self.num_layers)} for _ in range(self.
            batch_size) ]
254         partial_of_loss_wrt_params = {param : 0.0 for param in self.all_params}
255         bias_changes = {i : [0.0 for j in range( self.layers_config[i] ) ] for i in
            range(1, self.num_layers)}
256         for b in range(self.batch_size):
257             pred_err_backproped_at_layers[b][self.num_layers - 1] = [ y_errors[b] ]
258             for back_layer_index in reversed(range(1,self.num_layers)):
259                 input_vals = self.forw_prop_vals_at_layers[back_layer_index -1]
260                 deriv_sigmoids = self.gradient_vals_for_layers[back_layer_index]
261                 vars_in_layer = self.layer_vars[back_layer_index]
262                 vars_in_next_layer_back = self.layer_vars[back_layer_index - 1]
263                 vals_for_input_vars_dict = dict(zip(vars_in_next_layer_back, self.
                    forw_prop_vals_at_layers[back_layer_index - 1][b]))
264                 layer_params = self.layer_params[back_layer_index]
265                 transposed_layer_params = list(zip(*layer_params))
266                 for k,var1 in enumerate(vars_in_next_layer_back):
267                     for j,var2 in enumerate(vars_in_layer):
268                         pred_err_backproped_at_layers[b][back_layer_index - 1][k] = sum
                            ([self.vals_for_learnable_params[transposed_layer_params[k][
                                i]] * pred_err_backproped_at_layers[b][back_layer_index][i]
                                for i in range(len(vars_in_layer))])
269                 for j,var in enumerate(vars_in_layer):
270                     layer_params = self.layer_params[back_layer_index][j]
271                     input_vars_to_param_map = self.var_to_var_param[var]

```

```

272         param_to_vars_map = {param : var for var, param in
                               input_vars_to_param_map.items()}
273         for i,param in enumerate(layer_params):
274             partial_of_loss_wrt_params[param] +=
                pred_err_backproped_at_layers[b][back_layer_index][j] *
                vals_for_input_vars_dict[param_to_vars_map[param]] *
                deriv_sigmoids[b][j]
275         for k,var1 in enumerate(vars_in_next_layer_back):
276             for j,var2 in enumerate(vars_in_layer):
277                 if back_layer_index-1 > 0:
278                     bias_changes[back_layer_index-1][k] +=
                        pred_err_backproped_at_layers[b][back_layer_index - 1][k
                        ] * deriv_sigmoids[b][j]
279     for param in partial_of_loss_wrt_params:
280         partial_of_loss_wrt_param = partial_of_loss_wrt_params[param] / float(self.
            batch_size)
281
282     ##### Modification ----- Start ----- #####
283     self.m_weights[param] = self.beta_1 * self.m_weights[param] + (1 - self.
        beta_1) * partial_of_loss_wrt_param # First moment
284     self.v_weights[param] = self.beta_2 * self.v_weights[param] + (1 - self.
        beta_2) * partial_of_loss_wrt_param**2 # Second moment
285     step = self.learning_rate * self.v_weights[param]
286     m_hat = self.m_weights[param] / (1 - self.beta_1**self.timestep) # Bias
        corrected value for first moment
287     v_hat = self.v_weights[param] / (1 - self.beta_2**self.timestep) # Bias
        corrected value for second moment
288     step = self.learning_rate * m_hat / np.sqrt(v_hat + self.epsilon)
289     ##### Modification ----- End ----- #####
290
291     # step = self.learning_rate * partial_of_loss_wrt_param
292     self.vals_for_learnable_params[param] += step
293
294     for layer_index in range(1,self.num_layers):
295         for k in range(self.layers_config[layer_index]):
296
297             ##### Modification ----- Start ----- #####
298             # This modification updates bias using first and second moments
299             self.m_bias[layer_index][k] = self.beta_1 * self.m_bias[layer_index][k]
                + (1 - self.beta_1) * ( bias_changes[layer_index][k] / float(self.
                batch_size) )
300             self.v_bias[layer_index][k] = self.beta_2 * self.v_bias[layer_index][k]
                + (1 - self.beta_2) * ( bias_changes[layer_index][k] / float(self.
                batch_size) )**2 # Update velocity
301
302             m_hat = self.m_bias[layer_index][k] / (1 - self.beta_1**self.timestep) #
                Bias corrected value for first moment
303             v_hat = self.v_bias[layer_index][k] / (1 - self.beta_2**self.timestep) #
                Bias corrected value for second moment
304             self.bias[layer_index][k] += self.learning_rate * m_hat / (np.sqrt(v_hat
                + self.epsilon)) # Update bias
305     self.timestep += 1 # Increment time step
306     ##### Modification ----- End ----- #####
307
308     # self.bias[layer_index][k] += self.learning_rate * ( bias_changes[
        layer_index][k] / float(self.batch_size) )

```

2.2.3 Performance Analysis

For the multi-neuron case, there is a much more distinct difference between Adam and SGD+, as compared to the one-neuron case. When the learning rate is larger at 0.01 and 0.005, vanilla SGD performed the worst as it converges to a much higher loss compared to the other two models. This implies that SGD only converges to a local minimum, not the global. However, the curve of the SGD loss is the smoothest out of the three, meaning that its convergence behavior is the most stable. The SGD+ model performs better than SGD but worst than Adam as it converges to a loss value larger than that of the Adam optimizer. This again implies that SGD+ converges to a local minimum while Adam converges to the global minimum where loss is approximately 0.1. Finally, the Adam optimizer converges the quickest to the lowest final loss value. Thus, it is the best performing. However, its curve is the most jagged, which means the convergence behavior is the most unstable.

When the learning rate is at 0.001, SGD again had the worst performance. This time, SGD also had the most unstable convergence as its curve is the most jagged. The final loss value for SGD+ is slightly higher than Adam. The curve for Adam does not flatten out at 200 iterations. Thus, it is possible that the loss will continue to reduce with a larger number of iterations, reaching the global minimum loss of 0.1. Both SGD+ and Adam displayed stable convergence behavior as their curves are smooth.

Finally, when the learning rate is at 0.0001, the loss from SGD does not seem to converge but rather oscillate between the same constant value. On the other hand, SGD+ starts out with the highest loss and ends with a loss that is below SGD and slightly higher than Adam. Both curves for SGD+ and Adam seem to continue decreasing. Thus, the loss does not converge within 200 iterations.

Overall, the Adam optimizer outperformed the other two models. The ideal learning rate for the Adam optimizer seems to be 0.01, where the global minimum loss of 0.1 is obtained the quickest. However, there is a trade off between rate of convergence and stability. While the learning rate of 0.01 results in fast convergence, the converging behavior is unstable. On the other hand, with a smaller learning rate, the convergence is more stable but much slower.

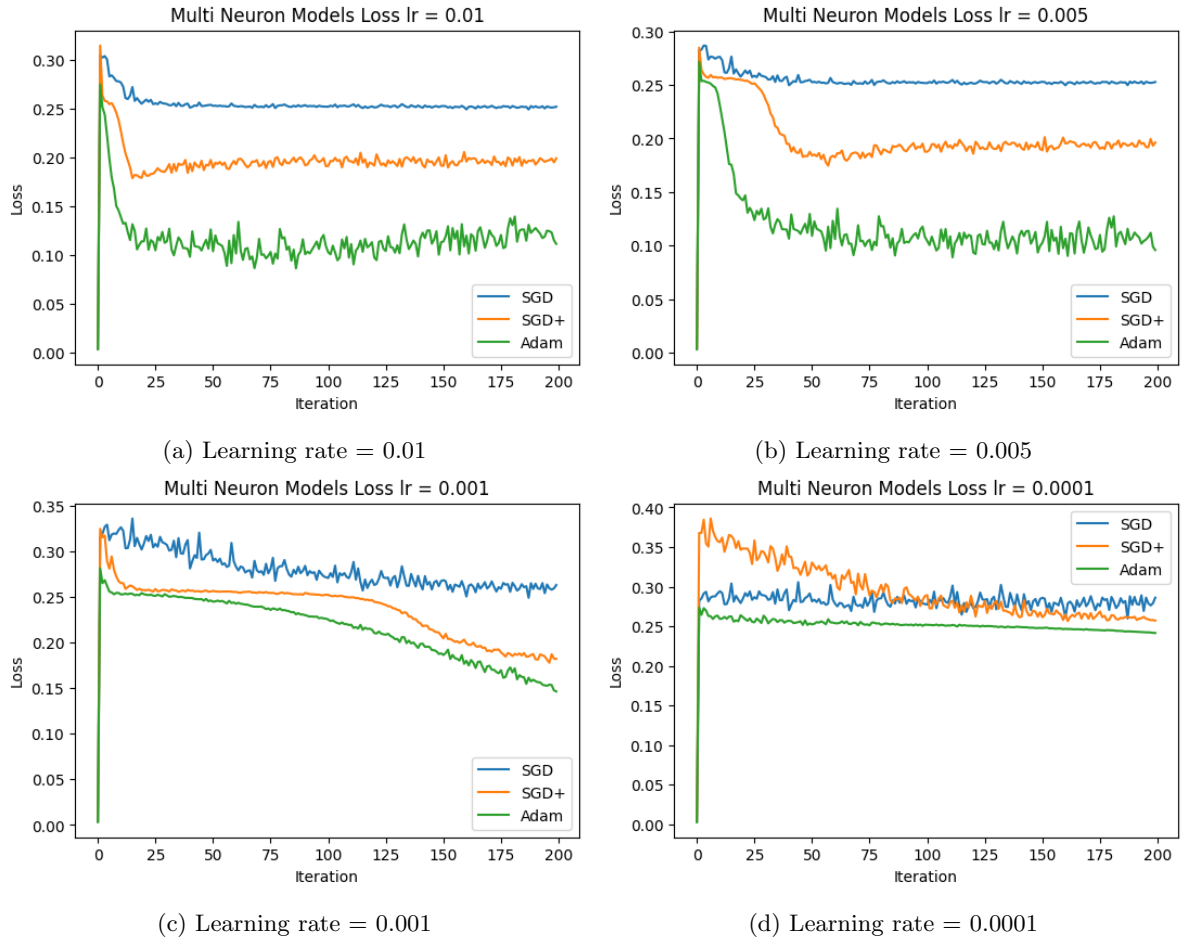


Figure 4: fig: Multi-neuron model comparisons

2.2.4 Adam Optimizer Hyperparameter

From table 2, we can see that the optimal hyperparameters are $\beta_1 = 0.8$ and $\beta_2 = 0.9$. The hyperparameter do not seem to have an impact on the time taken as all are very similar no matter the configuration.

Table 2: Comparison of Hyperparameters for Multi-neuron model

beta.1	beta.2	Time Taken	Final Loss	Minimum Loss
0.8	0.89	9.43007	0.231157	0.219362
0.8	0.9	9.31742	0.11348	0.0892691
0.8	0.95	9.08484	0.220015	0.212084
0.95	0.89	9.28071	0.156225	0.102485
0.95	0.9	9.20949	0.224975	0.207985
0.95	0.95	9.24699	0.196641	0.177626
0.99	0.89	9.28824	0.227094	0.194757
0.99	0.9	9.40451	0.214063	0.178864
0.99	0.95	9.19239	0.213802	0.183793

3 Extra Credit

3.1 Pixel Value Scaling and Normalization

Image data consists of integers ranging from 0 to 255. However, neural networks prefer input data to be floating-point numbers ranging from -1.0 to 1.0. To meet this requirement, pixel value scaling is first used to map the integer image data (0 to 255) to the floating-point range (0, 1.0). Then, pixel value normalization is used to map the scaled pixel values to span the floating-point range (-1.0, 1.0).

3.2 Comparing Performance with and without data normalization

In this section, we compare the performance between models that normalize data and models that dont. The without normalization version is created by simply deleting all lines that include **maxval** and replacing **batch_data = [item/maxval for item in batch_data]** with **batch_data = [item for item in batch_data]** .

Figure 5 shows the performance of the one-neuron and multi-neuron models with and without normalization. Without normalization, the performance of both models are significantly worst. For the one-neuron model, loss diverges. For the multi-neuron model, loss converges to a much higher value compared to the normalized one.

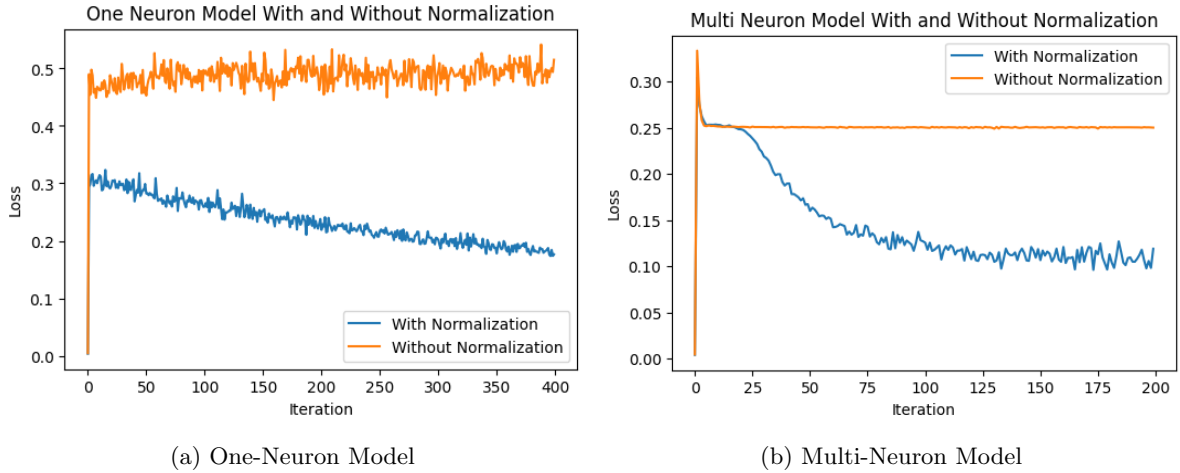


Figure 5: fig: Comparing Performance with and without Normalization

3.3 Comparing before and after truncation of data

A subclass called **OneNeuronSGDTruncated** is created. It inherits from the **OneNeuronSGD** class. To truncate the data, the function **gen_training_data** is modified. Mahalanobis distance is used to determine if the data is within five standard deviations.

```
1 class OneNeuronSGDTruncated(OneNeuronSGD):
2     def __init__(self, *args, ** kwargs):
3         super().__init__(*args, ** kwargs)
4
5     class AutogradCustomization(torch.nn.Module):
6         def __init__(self, cgp, num_samples_per_class):
7             super(ComputationalGraphPrimer.AutogradCustomization, self).__init__()
8             self.cgp = cgp
9             self.num_samples_per_class = num_samples_per_class
10
11         def gen_training_data(self):
12             mean1, mean2 = [3.0, 3.0], [5.0, 5.0]
13             covar1, covar2 = [[1.0, 0.0], [0.0, 1.0]], [[1.0, 0.0], [0.0, 1.0]]
14             inv_covar1 = np.linalg.inv(covar1)
15             inv_covar2 = np.linalg.inv(covar2)
16
17             def mahalanobis_dist(x, mean, inv_covar):
18                 return np.sqrt((x - mean).T @ inv_covar @ (x - mean))
19
20             data1 = []
21             data2 = []
22             for x in np.random.multivariate_normal(mean1, covar1, self.
23                 num_samples_per_class):
24                 if mahalanobis_dist(x, mean1, inv_covar1) <= 5:
25                     data1.append((list(x), 1))
26             for x in np.random.multivariate_normal(mean2, covar2, self.
27                 num_samples_per_class):
28                 if mahalanobis_dist(x, mean2, inv_covar2) <= 5:
29                     data2.append((list(x), 2))
30
31             training_data = data1 + data2
32             random.shuffle( training_data )
33             self.training_data = training_data
34
35 class MultiNeuronSGDTruncated(MultiNeuronSGD):
36     def __init__(self, *args, ** kwargs):
37         super().__init__(*args, ** kwargs)
38
39     class AutogradCustomization(torch.nn.Module):
40         def __init__(self, cgp, num_samples_per_class):
41             super(ComputationalGraphPrimer.AutogradCustomization, self).__init__()
42             self.cgp = cgp
43             self.num_samples_per_class = num_samples_per_class
44
45         def gen_training_data(self):
46             mean1, mean2 = [3.0, 3.0], [5.0, 5.0]
47             covar1, covar2 = [[1.0, 0.0], [0.0, 1.0]], [[1.0, 0.0], [0.0, 1.0]]
48             inv_covar1 = np.linalg.inv(covar1)
49             inv_covar2 = np.linalg.inv(covar2)
50
51             def mahalanobis_dist(x, mean, inv_covar):
52                 return np.sqrt((x - mean).T @ inv_covar @ (x - mean))
53
54             data1 = []
55             data2 = []
56             for x in np.random.multivariate_normal(mean1, covar1, self.
57                 num_samples_per_class):
58                 if mahalanobis_dist(x, mean1, inv_covar1) <= 5:
59                     data1.append((list(x), 1))
60             for x in np.random.multivariate_normal(mean2, covar2, self.
61                 num_samples_per_class):
62                 if mahalanobis_dist(x, mean2, inv_covar2) <= 5:
63                     data2.append((list(x), 2))
64
65             training_data = data1 + data2
66             random.shuffle( training_data )
67             self.training_data = training_data
```

Figure 6 shows the performance of the one-neuron and multi-neuron models with and without data truncation. The data is truncated to the interval $(\mu - 5\sigma, \mu + 5\sigma)$. For the non-truncated data, the final loss value is lower for both the one-neuron and multi-neuron models. However, the initial loss is lower when the data is truncated.

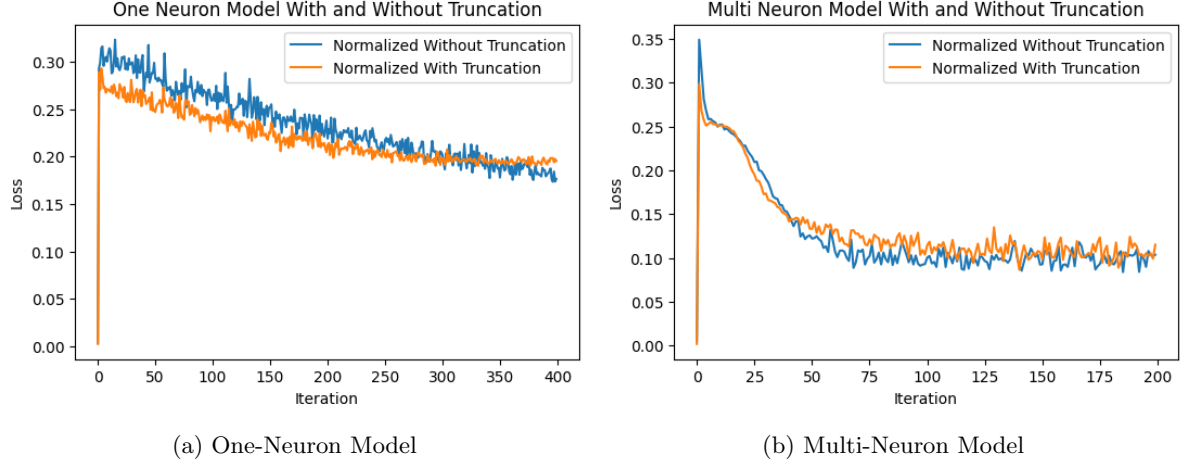


Figure 6: fig: Comparing Performance with and without Data Truncation

4 Source Code Main Task

```

1 import numpy as np
2 import random
3 import time
4 from tabulate import tabulate
5 import sys
6
7 seed = 0
8 random.seed(seed)
9 np.random.seed(seed)
10
11 from ComputationalGraphPrimer import *
12
13 class OneNeuronSGD(ComputationalGraphPrimer):
14     ##### Modification ----- Start ----- #####
15     # The OneNeuronSGD is created to be a subclass of the ComputationalGraphPrimer class
16     # super() is used to inherit functions from the CGP class
17     def __init__(self, *args, ** kwargs):
18         super().__init__(*args, ** kwargs) # Inherit from CGP class
19     ##### Modification ----- End ----- #####
20
21     def run_training_loop_one_neuron_model(self, training_data):
22         self.vals_for_learnable_params = {param: random.uniform(0,1) for param in self.
23             learnable_params}
24         self.bias = random.uniform(0,1)
25         class DataLoader:
26             def __init__(self, training_data, batch_size):
27                 self.training_data = training_data
28                 self.batch_size = batch_size
29                 self.class_0_samples = [(item, 0) for item in self.training_data[0]]
30                 self.class_1_samples = [(item, 1) for item in self.training_data[1]]
31             def __len__(self):
32                 return len(self.training_data[0]) + len(self.training_data[1])
33             def __getitem__(self):
34                 cointoss = random.choice([0,1])
35                 if cointoss == 0:
36                     return random.choice(self.class_0_samples)
37                 else:
38                     return random.choice(self.class_1_samples)
39             def getbatch(self):
40                 batch_data, batch_labels = [], []
41                 maxval = 0.0

```

```

41         for _ in range(self.batch_size):
42             item = self._getitem()
43             if np.max(item[0]) > maxval:
44                 maxval = np.max(item[0])
45                 batch_data.append(item[0])
46                 batch_labels.append(item[1])
47             batch_data = [item/maxval for item in batch_data]
48             batch = [batch_data, batch_labels]
49             return batch
50     data_loader = DataLoader(training_data, batch_size=self.batch_size)
51     loss_running_record = []
52     i = 0
53     avg_loss_over_iterations = 0.0
54     for i in range(self.training_iterations):
55         data = data_loader.getbatch()
56         data_tuples_in_batch = data[0]
57         class_labels_in_batch = data[1]
58         y_preds, deriv_sigmoids = self.forward_prop_one_neuron_model(
59             data_tuples_in_batch)
60         loss = sum([(abs(class_labels_in_batch[i] - y_preds[i]))**2 for i in range(
61             len(class_labels_in_batch))])
62         avg_loss_over_iterations += loss / float(len(class_labels_in_batch))
63         if i%(self.display_loss_how_often) == 0:
64             avg_loss_over_iterations /= self.display_loss_how_often
65             loss_running_record.append(avg_loss_over_iterations)
66             print("[iter=%d] loss=%.4f" % (i+1, avg_loss_over_iterations))
67             avg_loss_over_iterations = 0.0
68             y_errors_in_batch = list(map(operator.sub, class_labels_in_batch, y_preds))
69             self.backprop_and_update_params_one_neuron_model(data_tuples_in_batch,
70                 y_preds, y_errors_in_batch, deriv_sigmoids)
71     # Remove plots
72     return loss_running_record # return loss record
73
74 class OneNeuronSGDPlus(ComputationalGraphPrimer):
75     ##### Modification ----- Start ----- #####
76     # The OneNeuronSGDPlus is created to be a subclass of the ComputationalGraphPrimer
77     class
78     # super() is used to inherit functions from the CGP class
79     def __init__(self, beta, *args, **kwargs): # Add beta to __init__ parameter
80         super().__init__(*args, **kwargs) # Inherit from CGP class
81         self.beta = beta # Initialize Momentum coefficient
82     ##### Modification ----- End ----- #####
83
84     def run_training_loop_one_neuron_model(self, training_data):
85         self.vals_for_learnable_params = {param: random.uniform(0,1) for param in self.
86             learnable_params}
87         self.bias = random.uniform(0,1)
88
89         ##### Modification ----- Start ----- #####
90         # Initialize velocity (accumulated gradient) for weights and biases
91         # Velocity will be used for momentum update calculations
92         self.v_weights = {param: 0.0 for param in self.learnable_params} # Set all
93             weights to 0
94         self.v_bias = 0 # Initialize bias as 0
95         ##### Modification ----- End ----- #####
96
97     class DataLoader:
98         def __init__(self, training_data, batch_size):
99             self.training_data = training_data
100             self.batch_size = batch_size
101             self.class_0_samples = [(item, 0) for item in self.training_data[0]]
102             self.class_1_samples = [(item, 1) for item in self.training_data[1]]
103         def __len__(self):
104             return len(self.training_data[0]) + len(self.training_data[1])
105         def _getitem(self):
106             cointoss = random.choice([0,1])
107             if cointoss == 0:
108                 return random.choice(self.class_0_samples)
109             else:
110                 return random.choice(self.class_1_samples)
111         def getbatch(self):
112             batch_data, batch_labels = [], []
113             maxval = 0.0

```

```

108         for _ in range(self.batch_size):
109             item = self._getitem()
110             if np.max(item[0]) > maxval:
111                 maxval = np.max(item[0])
112                 batch_data.append(item[0])
113                 batch_labels.append(item[1])
114             batch_data = [item/maxval for item in batch_data]
115             batch = [batch_data, batch_labels]
116         return batch
117     data_loader = DataLoader(training_data, batch_size=self.batch_size)
118     loss_running_record = []
119     i = 0
120     avg_loss_over_iterations = 0.0
121     for i in range(self.training_iterations):
122         data = data_loader.getbatch()
123         data_tuples_in_batch = data[0]
124         class_labels_in_batch = data[1]
125         y_preds, deriv_sigmoids = self.forward_prop_one_neuron_model(
126             data_tuples_in_batch)
127         loss = sum([(abs(class_labels_in_batch[i] - y_preds[i]))**2 for i in range(
128             len(class_labels_in_batch))])
129         avg_loss_over_iterations += loss / float(len(class_labels_in_batch))
130         if i%(self.display_loss_how_often) == 0:
131             avg_loss_over_iterations /= self.display_loss_how_often
132             loss_running_record.append(avg_loss_over_iterations)
133             print("[iter=%d] loss=%.4f" % (i+1, avg_loss_over_iterations))
134             avg_loss_over_iterations = 0.0
135             y_errors_in_batch = list(map(operator.sub, class_labels_in_batch, y_preds))
136             self.backprop_and_update_params_one_neuron_model(data_tuples_in_batch,
137                 y_preds, y_errors_in_batch, deriv_sigmoids)
138         # Remove plots
139         return loss_running_record # return loss record
140
141     def backprop_and_update_params_one_neuron_model(self, data_tuples_in_batch,
142         predictions, y_errors_in_batch, deriv_sigmoids):
143         input_vars = self.independent_vars
144         input_vars_to_param_map = self.var_to_var_param[self.output_vars[0]]
145         param_to_vars_map = {param : var for var, param in input_vars_to_param_map.items
146             ()}
147         vals_for_learnable_params = self.vals_for_learnable_params
148         for i,param in enumerate(self.vals_for_learnable_params):
149             partial_of_loss_wrt_param = 0.0
150             for j in range(self.batch_size):
151                 vals_for_input_vars_dict = dict(zip(input_vars, list(
152                     data_tuples_in_batch[j])))
153                 partial_of_loss_wrt_param += - y_errors_in_batch[j] *
154                     vals_for_input_vars_dict[param_to_vars_map[param]] * deriv_sigmoids[
155                         j]
156             partial_of_loss_wrt_param /= float(self.batch_size)
157
158             ##### Modification ----- Start ----- #####
159             # This modification updates the learnable parameters using momentum
160             self.v_weights[param]= self.beta * self.v_weights[param] +
161                 partial_of_loss_wrt_param # Update velocity (accumulated gradient)
162             step = -self.learning_rate * self.v_weights[param] # Step size = learning
163                 rate * velocity
164             ##### Modification ----- End ----- #####
165
166             # step = self.learning_rate * partial_of_loss_wrt_param
167             self.vals_for_learnable_params[param] += step
168
169         y_error_avg = sum(y_errors_in_batch) / float(self.batch_size)
170         deriv_sigmoid_avg = sum(deriv_sigmoids) / float(self.batch_size)
171
172         ##### Modification ----- Start ----- #####
173         # This modification updates the biases using momentum
174         self.v_bias = self.beta * self.v_bias + y_error_avg * deriv_sigmoid_avg #
175             Update velocity
176         self.bias += self.learning_rate * self.v_bias # Update bias
177         ##### Modification ----- End ----- #####
178         # self.bias += self.learning_rate * y_error_avg * deriv_sigmoid_avg
179
180     class OneNeuronSGDAdam(ComputationalGraphPrimer):

```

```

170 ##### Modification ----- Start ----- #####
171 # The OneNeuronSGDPlus is created to be a subclass of the ComputationalGraphPrimer
    class
172 # super() is used to inherit functions from the CGP class
173 def __init__(self, beta_1, beta_2, epsilon, *args, **kwargs):
174     super().__init__(*args, **kwargs) # Inherit from CGP class
175     self.beta_1 = beta_1 # Initialize decay rate for first moment
176     self.beta_2 = beta_2 # Initialize decay rate for second moment
177     self.epsilon = epsilon
178 ##### Modification ----- End ----- #####
179
180 def run_training_loop_one_neuron_model(self, training_data):
181     self.vals_for_learnable_params = {param: random.uniform(0,1) for param in self.
        learnable_params}
182     self.bias = random.uniform(0,1)
183
184 ##### Modification ----- Start ----- #####
185 # Initialize first moment (m) and second moment (v) for both weights and bias
186 self.v_weights = {param: 0.0 for param in self.learnable_params}
187 self.v_bias = 0
188 self.m_weights = {param: 0.0 for param in self.learnable_params}
189 self.m_bias = 0
190 self.timestep = 1 # Initialize time step for calculation of bias corected values
191 ##### Modification ----- End ----- #####
192
193 class DataLoader:
194     def __init__(self, training_data, batch_size):
195         self.training_data = training_data
196         self.batch_size = batch_size
197         self.class_0_samples = [(item, 0) for item in self.training_data[0]]
198         self.class_1_samples = [(item, 1) for item in self.training_data[1]]
199     def __len__(self):
200         return len(self.training_data[0]) + len(self.training_data[1])
201     def _getitem(self):
202         cointoss = random.choice([0,1])
203         if cointoss == 0:
204             return random.choice(self.class_0_samples)
205         else:
206             return random.choice(self.class_1_samples)
207     def getbatch(self):
208         batch_data, batch_labels = [], []
209         maxval = 0.0
210         for _ in range(self.batch_size):
211             item = self._getitem()
212             if np.max(item[0]) > maxval:
213                 maxval = np.max(item[0])
214             batch_data.append(item[0])
215             batch_labels.append(item[1])
216         batch_data = [item/maxval for item in batch_data]
217         batch = [batch_data, batch_labels]
218         return batch
219 data_loader = DataLoader(training_data, batch_size=self.batch_size)
220 loss_running_record = []
221 i = 0
222 avg_loss_over_iterations = 0.0
223 for i in range(self.training_iterations):
224     data = data_loader.getbatch()
225     data_tuples_in_batch = data[0]
226     class_labels_in_batch = data[1]
227     y_preds, deriv_sigmoids = self.forward_prop_one_neuron_model(
        data_tuples_in_batch)
228     loss = sum([(abs(class_labels_in_batch[i] - y_preds[i]))*2 for i in range(
        len(class_labels_in_batch))])
229     avg_loss_over_iterations += loss / float(len(class_labels_in_batch))
230     if i%(self.display_loss_how_often) == 0:
231         avg_loss_over_iterations /= self.display_loss_how_often
232         loss_running_record.append(avg_loss_over_iterations)
233         print("[iter=%d] loss=%0.4f" % (i+1, avg_loss_over_iterations))
234         avg_loss_over_iterations = 0.0
235     y_errors_in_batch = list(map(operator.sub, class_labels_in_batch, y_preds))
236     self.backprop_and_update_params_one_neuron_model(data_tuples_in_batch,
        y_preds, y_errors_in_batch, deriv_sigmoids)
237 # Remove plots

```

```

238         return loss_running_record # return loss record
239
240     def backprop_and_update_params_one_neuron_model(self, data_tuples_in_batch,
241 predictions, y_errors_in_batch, deriv_sigmoids):
242         input_vars = self.independent_vars
243         input_vars_to_param_map = self.var_to_var_param[self.output_vars[0]]
244         param_to_vars_map = {param : var for var, param in input_vars_to_param_map.items
245                             ()}
246         vals_for_learnable_params = self.vals_for_learnable_params
247         for i,param in enumerate(self.vals_for_learnable_params):
248             partial_of_loss_wrt_param = 0.0
249             for j in range(self.batch_size):
250                 vals_for_input_vars_dict = dict(zip(input_vars, list(
251                     data_tuples_in_batch[j])))
252                 partial_of_loss_wrt_param += - y_errors_in_batch[j] *
253                     vals_for_input_vars_dict[param_to_vars_map[param]] * deriv_sigmoids[
254                         j]
255             partial_of_loss_wrt_param /= float(self.batch_size)
256             ##### Modification ----- Start ----- #####
257             # This modification updates weighs using first and second moments
258             self.m_weights[param] = self.beta_1 * self.m_weights[param] + (1 - self.
259                 beta_1) * partial_of_loss_wrt_param # First moment
260             self.v_weights[param] = self.beta_2 * self.v_weights[param] + (1 - self.
261                 beta_2) * partial_of_loss_wrt_param**2 # Second moment
262             m_hat = self.m_weights[param] / (1 - self.beta_1**self.timestep) # Bias
263                 corrected value for first moment
264             v_hat = self.v_weights[param] / (1 - self.beta_2**self.timestep) # Bias
265                 corrected value for second moment
266             step = -self.learning_rate * m_hat / np.sqrt(v_hat + self.epsilon)
267             ##### Modification ----- End ----- #####
268
269             # step = self.learning_rate * partial_of_loss_wrt_param
270             self.vals_for_learnable_params[param] += step
271
272         y_error_avg = sum(y_errors_in_batch) / float(self.batch_size)
273         deriv_sigmoid_avg = sum(deriv_sigmoids) / float(self.batch_size)
274
275         ##### Modification ----- Start ----- #####
276         # This modification updates bias using first and second moments
277         self.m_bias = self.beta_1 * self.m_bias + (1 - self.beta_1) * y_error_avg *
278             deriv_sigmoid_avg
279         self.v_bias = self.beta_2 * self.v_bias + (1 - self.beta_2) * (y_error_avg *
280             deriv_sigmoid_avg)**2 # Update velocity
281
282         m_hat = self.m_bias / (1 - self.beta_1**self.timestep) # Bias corrected value
283             for first moment
284         v_hat = self.v_bias / (1 - self.beta_2**self.timestep) # Bias corrected value
285             for second moment
286         self.bias += self.learning_rate * m_hat / (np.sqrt(v_hat + self.epsilon)) #
287             Update bias
288         self.timestep += 1 # Increment time step
289         ##### Modification ----- End ----- #####
290
291         # self.bias += self.learning_rate * y_error_avg * deriv_sigmoid_avg
292
293     class MultiNeuronSGD(ComputationalGraphPrimer):
294         ##### Modification ----- Start ----- #####
295         # The MultiNeuronSGD is created to be a subclass of the ComputationalGraphPrimer
296         class
297         # super() is used to inherit functions from the CGP class
298         def __init__(self, *args, ** kwargs):
299             super().__init__(*args, ** kwargs) # Inherit from CGP class
300         ##### Modification ----- End ----- #####
301
302     def run_training_loop_multi_neuron_model(self, training_data):
303         class DataLoader:
304             def __init__(self, training_data, batch_size):
305                 self.training_data = training_data
306                 self.batch_size = batch_size
307                 self.class_0_samples = [(item, 0) for item in self.training_data[0]]
308                 self.class_1_samples = [(item, 1) for item in self.training_data[1]]
309             def __len__(self):
310                 return len(self.training_data[0]) + len(self.training_data[1])

```

```

296         def _getitem(self):
297             cointoss = random.choice([0,1])
298             if cointoss == 0:
299                 return random.choice(self.class_0_samples)
300             else:
301                 return random.choice(self.class_1_samples)
302         def getbatch(self):
303             batch_data, batch_labels = [], []
304             maxval = 0.0
305             for _ in range(self.batch_size):
306                 item = self._getitem()
307                 if np.max(item[0]) > maxval:
308                     maxval = np.max(item[0])
309                 batch_data.append(item[0])
310                 batch_labels.append(item[1])
311             batch_data = [item/maxval for item in batch_data]
312             batch = [batch_data, batch_labels]
313             return batch
314         self.vals_for_learnable_params = {param: random.uniform(0,1) for param in self.
            learnable_params}
315         self.bias = {i : [random.uniform(0,1) for j in range( self.layers_config[i] )
            ] for i in range(1, self.num_layers)}
316         data_loader = DataLoader(training_data, batch_size=self.batch_size)
317         loss_running_record = []
318         i = 0
319         avg_loss_over_iterations = 0.0
320         for i in range(self.training_iterations):
321             data = data_loader.getbatch()
322             data_tuples = data[0]
323             class_labels = data[1]
324             self.forward_prop_multi_neuron_model(data_tuples)
325             predicted_labels_for_batch = self.forw_prop_vals_at_layers[self.num_layers
                -1]
326             y_preds = [item for sublist in predicted_labels_for_batch for item in
                sublist]
327             loss = sum([(abs(class_labels[i] - y_preds[i]))**2 for i in range(len(
                class_labels))])
328             loss_avg = loss / float(len(class_labels))
329             avg_loss_over_iterations += loss_avg
330             if i%(self.display_loss_how_often) == 0:
331                 avg_loss_over_iterations /= self.display_loss_how_often
332                 loss_running_record.append(avg_loss_over_iterations)
333                 print("[iter=%d] loss=%.4f" % (i+1, avg_loss_over_iterations))
334                 avg_loss_over_iterations = 0.0
335             y_errors_in_batch = list(map(operator.sub, class_labels, y_preds))
336             self.backprop_and_update_params_multi_neuron_model(y_preds,
                y_errors_in_batch)
337         # Remove plots
338         return loss_running_record
339
340     class MultiNeuronSGDPlus(ComputationalGraphPrimer):
341         ##### Modification ----- Start ----- #####
342         # The MultiNeuronSGDPlus is created to be a subclass of the ComputationalGraphPrimer
            class
343         # super() is used to inherit functions from the CGP class
344         def __init__(self, beta, *args, ** kwargs):
345             super().__init__(*args, ** kwargs) # Inherit from CGP class
346             self.beta = beta # Initialize Momentum coefficient
347         ##### Modification ----- End ----- #####
348
349         def run_training_loop_multi_neuron_model(self, training_data):
350             class DataLoader:
351                 def __init__(self, training_data, batch_size):
352                     self.training_data = training_data
353                     self.batch_size = batch_size
354                     self.class_0_samples = [(item, 0) for item in self.training_data[0]]
355                     self.class_1_samples = [(item, 1) for item in self.training_data[1]]
356                 def __len__(self):
357                     return len(self.training_data[0]) + len(self.training_data[1])
358                 def _getitem(self):
359                     cointoss = random.choice([0,1])
360                     if cointoss == 0:
361                         return random.choice(self.class_0_samples)

```

```

362         else:
363             return random.choice(self.class_1_samples)
364     def getbatch(self):
365         batch_data, batch_labels = [], []
366         maxval = 0.0
367         for _ in range(self.batch_size):
368             item = self._getitem()
369             if np.max(item[0]) > maxval:
370                 maxval = np.max(item[0])
371                 batch_data.append(item[0])
372                 batch_labels.append(item[1])
373             batch_data = [item/maxval for item in batch_data]
374             batch = [batch_data, batch_labels]
375             return batch
376     self.vals_for_learnable_params = {param: random.uniform(0,1) for param in self.
        learnable_params}
377     self.bias = {i : [random.uniform(0,1) for j in range( self.layers_config[i] )
        ] for i in range(1, self.num_layers)}
378
379     ##### Modification ----- Start ----- #####
380     # Initialize velocity (accumulated gradient) for weights and biases
381     # Velocity will be used for momentum update calculations
382     self.v_weights = {param: 0.0 for param in self.learnable_params} # Set all
        weights to 0
383     self.v_bias = {i: [0.0 for _ in range(self.layers_config[i])] for i in range(1,
        self.num_layers)} # Initialize bias as 0
384     ##### Modification ----- End ----- #####
385
386     data_loader = DataLoader(training_data, batch_size=self.batch_size)
387     loss_running_record = []
388     i = 0
389     avg_loss_over_iterations = 0.0
390     for i in range(self.training_iterations):
391         data = data_loader.getbatch()
392         data_tuples = data[0]
393         class_labels = data[1]
394         self.forward_prop_multi_neuron_model(data_tuples)
395         predicted_labels_for_batch = self.forw_prop_vals_at_layers[self.num_layers
            -1]
396         y_preds = [item for sublist in predicted_labels_for_batch for item in
            sublist]
397         loss = sum([(abs(class_labels[i] - y_preds[i]))**2 for i in range(len(
            class_labels))])
398         loss_avg = loss / float(len(class_labels))
399         avg_loss_over_iterations += loss_avg
400         if i%(self.display_loss_how_often) == 0:
401             avg_loss_over_iterations /= self.display_loss_how_often
402             loss_running_record.append(avg_loss_over_iterations)
403             print("[iter=%d] loss=%.4f" % (i+1, avg_loss_over_iterations))
404             avg_loss_over_iterations = 0.0
405             y_errors_in_batch = list(map(operator.sub, class_labels, y_preds))
406             self.backprop_and_update_params_multi_neuron_model(y_preds,
                y_errors_in_batch)
407         # Remove plots
408         return loss_running_record
409
410     def backprop_and_update_params_multi_neuron_model(self, predictions, y_errors):
411         pred_err_backproped_at_layers = [ {i : [None for j in range( self.
            layers_config[i] ) ] for i in range(self.num_layers)} for _ in range(self.
            batch_size) ]
412         partial_of_loss_wrt_params = {param : 0.0 for param in self.all_params}
413         bias_changes = {i : [0.0 for j in range( self.layers_config[i] ) ] for i in
            range(1, self.num_layers)}
414         for b in range(self.batch_size):
415             pred_err_backproped_at_layers[b][self.num_layers - 1] = [ y_errors[b] ]
416             for back_layer_index in reversed(range(1, self.num_layers)):
417                 input_vals = self.forw_prop_vals_at_layers[back_layer_index -1]
418                 deriv_sigmoids = self.gradient_vals_for_layers[back_layer_index]
419                 vars_in_layer = self.layer_vars[back_layer_index]
420                 vars_in_next_layer_back = self.layer_vars[back_layer_index - 1]
421                 vals_for_input_vars_dict = dict(zip(vars_in_next_layer_back, self.
                    forw_prop_vals_at_layers[back_layer_index - 1][b]))
422                 layer_params = self.layer_params[back_layer_index]

```

```

423         transposed_layer_params = list(zip(*layer_params))
424         for k,var1 in enumerate(vars_in_next_layer_back):
425             for j,var2 in enumerate(vars_in_layer):
426                 pred_err_backproped_at_layers[b][back_layer_index - 1][k] = sum
                     ([self.vals_for_learnable_params[transposed_layer_params[k][
                     i]] * pred_err_backproped_at_layers[b][back_layer_index][i]
                     for i in range(len(vars_in_layer))])
427         for j,var in enumerate(vars_in_layer):
428             layer_params = self.layer_params[back_layer_index][j]
429             input_vars_to_param_map = self.var_to_var_param[var]
430             param_to_vars_map = {param : var for var, param in
                     input_vars_to_param_map.items()}
431             for i,param in enumerate(layer_params):
432                 partial_of_loss_wrt_params[param] +=
                     pred_err_backproped_at_layers[b][back_layer_index][j] *
                     vals_for_input_vars_dict[param_to_vars_map[param]] *
                     deriv_sigmoids[b][j]
433         for k,var1 in enumerate(vars_in_next_layer_back):
434             for j,var2 in enumerate(vars_in_layer):
435                 if back_layer_index-1 > 0:
436                     bias_changes[back_layer_index-1][k] +=
                     pred_err_backproped_at_layers[b][back_layer_index - 1][k]
                     ] * deriv_sigmoids[b][j]
437     for param in partial_of_loss_wrt_params:
438         partial_of_loss_wrt_param = partial_of_loss_wrt_params[param] / float(self.
         batch_size)
439
440     ##### Modification ----- Start ----- #####
441     self.v_weights[param] = self.beta * self.v_weights[param] +
         partial_of_loss_wrt_param
442     step = self.learning_rate * self.v_weights[param]
443     ##### Modification ----- End ----- #####
444
445     # step = self.learning_rate * partial_of_loss_wrt_param
446     self.vals_for_learnable_params[param] += step
447     for layer_index in range(1,self.num_layers):
448         for k in range(self.layers_config[layer_index]):
449             ##### Modification ----- Start ----- #####
450             self.v_bias[layer_index][k] = self.beta * self.v_bias[layer_index][k] +
                 ( bias_changes[layer_index][k] / float(self.batch_size) ) # Update
                 momentum
451             self.bias[layer_index][k] += self.learning_rate * self.v_bias[
                 layer_index][k] # Update Bias
452             ##### Modification ----- End ----- #####
453
454             # self.bias[layer_index][k] += self.learning_rate * ( bias_changes[
                 layer_index][k] / float(self.batch_size) )
455
456
457     class MultiNeuronSGDAdam(ComputationalGraphPrimer):
458         ##### Modification ----- Start ----- #####
459         # The MultiNeuronSGDAdam is created to be a subclass of the ComputationalGraphPrimer
         class
460         # super() is used to inherit functions from the CGP class
461         def __init__(self, beta_1, beta_2, epsilon, *args, ** kwargs):
462             super().__init__(*args, ** kwargs) # Inherit from CGP class
463             self.beta_1 = beta_1 # Initialize decay rate for first moment
464             self.beta_2 = beta_2 # Initialize decay rate for second moment
465             self.epsilon = epsilon
466         ##### Modification ----- End ----- #####
467
468     def run_training_loop_multi_neuron_model(self, training_data):
469         class DataLoader:
470             def __init__(self, training_data, batch_size):
471                 self.training_data = training_data
472                 self.batch_size = batch_size
473                 self.class_0_samples = [(item, 0) for item in self.training_data[0]]
474                 self.class_1_samples = [(item, 1) for item in self.training_data[1]]
475             def __len__(self):
476                 return len(self.training_data[0]) + len(self.training_data[1])
477             def _getitem(self):
478                 cointoss = random.choice([0,1])
479                 if cointoss == 0:

```

```

480         return random.choice(self.class_0_samples)
481     else:
482         return random.choice(self.class_1_samples)
483 def getbatch(self):
484     batch_data, batch_labels = [], []
485     maxval = 0.0
486     for _ in range(self.batch_size):
487         item = self._getitem()
488         if np.max(item[0]) > maxval:
489             maxval = np.max(item[0])
490         batch_data.append(item[0])
491         batch_labels.append(item[1])
492     batch_data = [item/maxval for item in batch_data]
493     batch = [batch_data, batch_labels]
494     return batch
495 self.vals_for_learnable_params = {param: random.uniform(0,1) for param in self.
    learnable_params}
496 self.bias = {i : [random.uniform(0,1) for j in range( self.layers_config[i] )
    ] for i in range(1, self.num_layers)}
497
498 ##### Modification ----- Start ----- #####
499 # Initialize first moment (m) and second moment (v) for both weights and bias
500 self.v_weights = {param: 0.0 for param in self.learnable_params}
501 self.v_bias = {i: [0.0 for _ in range(self.layers_config[i])] for i in range(1,
    self.num_layers)}
502 self.m_weights = {param: 0.0 for param in self.learnable_params}
503 self.m_bias = {i: [0.0 for _ in range(self.layers_config[i])] for i in range(1,
    self.num_layers)}
504 self.timestep = 1 # Initialize time step for calculation of bias corected values
505 ##### Modification ----- End ----- #####
506
507 data_loader = DataLoader(training_data, batch_size=self.batch_size)
508 loss_running_record = []
509 i = 0
510 avg_loss_over_iterations = 0.0
511 for i in range(self.training_iterations):
512     data = data_loader.getbatch()
513     data_tuples = data[0]
514     class_labels = data[1]
515     self.forward_prop_multi_neuron_model(data_tuples)
516     predicted_labels_for_batch = self.forw_prop_vals_at_layers[self.num_layers
    -1]
517     y_preds = [item for sublist in predicted_labels_for_batch for item in
    sublist]
518     loss = sum([(abs(class_labels[i] - y_preds[i]))**2 for i in range(len(
    class_labels))])
519     loss_avg = loss / float(len(class_labels))
520     avg_loss_over_iterations += loss_avg
521     if i%(self.display_loss_how_often) == 0:
522         avg_loss_over_iterations /= self.display_loss_how_often
523         loss_running_record.append(avg_loss_over_iterations)
524         print("[iter=%d] loss_u=%u.4f" % (i+1, avg_loss_over_iterations))
525         avg_loss_over_iterations = 0.0
526     y_errors_in_batch = list(map(operator.sub, class_labels, y_preds))
527     self.backprop_and_update_params_multi_neuron_model(y_preds,
    y_errors_in_batch)
528 # Remove plots
529 return loss_running_record
530
531 def backprop_and_update_params_multi_neuron_model(self, predictions, y_errors):
532     pred_err_backproped_at_layers = [ {i : [None for j in range( self.
    layers_config[i] ) ] for i in range(self.num_layers)} for _ in range(self.
    batch_size) ]
533     partial_of_loss_wrt_params = {param : 0.0 for param in self.all_params}
534     bias_changes = {i : [0.0 for j in range( self.layers_config[i] ) ] for i in
    range(1, self.num_layers)}
535     for b in range(self.batch_size):
536         pred_err_backproped_at_layers[b][self.num_layers - 1] = [ y_errors[b] ]
537         for back_layer_index in reversed(range(1, self.num_layers)):
538             input_vals = self.forw_prop_vals_at_layers[back_layer_index -1]
539             deriv_sigmoids = self.gradient_vals_for_layers[back_layer_index]
540             vars_in_layer = self.layer_vars[back_layer_index]
541             vars_in_next_layer_back = self.layer_vars[back_layer_index - 1]

```

```

542         vals_for_input_vars_dict = dict(zip(vars_in_next_layer_back, self.
543             forw_prop_vals_at_layers[back_layer_index - 1][b]))
544         layer_params = self.layer_params[back_layer_index]
545         transposed_layer_params = list(zip(*layer_params))
546         for k,var1 in enumerate(vars_in_next_layer_back):
547             for j,var2 in enumerate(vars_in_layer):
548                 pred_err_backproped_at_layers[b][back_layer_index - 1][k] = sum
549                     ([self.vals_for_learnable_params[transposed_layer_params[k][
550                         i]] * pred_err_backproped_at_layers[b][back_layer_index][i]
551                     for i in range(len(vars_in_layer))])
552             for j,var in enumerate(vars_in_layer):
553                 layer_params = self.layer_params[back_layer_index][j]
554                 input_vars_to_param_map = self.var_to_var_param[var]
555                 param_to_vars_map = {param : var for var, param in
556                     input_vars_to_param_map.items()}
557                 for i,param in enumerate(layer_params):
558                     partial_of_loss_wrt_params[param] +=
559                         pred_err_backproped_at_layers[b][back_layer_index][j] *
560                         vals_for_input_vars_dict[param_to_vars_map[param]] *
561                         deriv_sigmoids[b][j]
562             for k,var1 in enumerate(vars_in_next_layer_back):
563                 for j,var2 in enumerate(vars_in_layer):
564                     if back_layer_index-1 > 0:
565                         bias_changes[back_layer_index-1][k] +=
566                             pred_err_backproped_at_layers[b][back_layer_index - 1][k]
567                             * deriv_sigmoids[b][j]
568         for param in partial_of_loss_wrt_params:
569             partial_of_loss_wrt_param = partial_of_loss_wrt_params[param] / float(self.
570                 batch_size)
571
572         ##### Modification ----- Start ----- #####
573         self.m_weights[param] = self.beta_1 * self.m_weights[param] + (1 - self.
574             beta_1) * partial_of_loss_wrt_param # First moment
575         self.v_weights[param] = self.beta_2 * self.v_weights[param] + (1 - self.
576             beta_2) * partial_of_loss_wrt_param**2 # Second moment
577         step = self.learning_rate * self.v_weights[param]
578         m_hat = self.m_weights[param] / (1 - self.beta_1**self.timestep) # Bias
579             corrected value for first moment
580         v_hat = self.v_weights[param] / (1 - self.beta_2**self.timestep) # Bias
581             corrected value for second moment
582         step = self.learning_rate * m_hat / np.sqrt(v_hat + self.epsilon)
583         ##### Modification ----- End ----- #####
584
585         # step = self.learning_rate * partial_of_loss_wrt_param
586         self.vals_for_learnable_params[param] += step
587
588     for layer_index in range(1,self.num_layers):
589         for k in range(self.layers_config[layer_index]):
590
591             ##### Modification ----- Start ----- #####
592             # This modification updates bias using first and second moments
593             self.m_bias[layer_index][k] = self.beta_1 * self.m_bias[layer_index][k]
594                 + (1 - self.beta_1) * ( bias_changes[layer_index][k] / float(self.
595                     batch_size) )
596             self.v_bias[layer_index][k] = self.beta_2 * self.v_bias[layer_index][k]
597                 + (1 - self.beta_2) * ( bias_changes[layer_index][k] / float(self.
598                     batch_size) )**2 # Update velocity
599
600             m_hat = self.m_bias[layer_index][k] / (1 - self.beta_1**self.timestep) #
601                 Bias corrected value for first moment
602             v_hat = self.v_bias[layer_index][k] / (1 - self.beta_2**self.timestep) #
603                 Bias corrected value for second moment
604             self.bias[layer_index][k] += self.learning_rate * m_hat / (np.sqrt(v_hat
605                 + self.epsilon)) # Update bias
606         self.timestep += 1 # Increment time step
607         ##### Modification ----- End ----- #####
608
609         # self.bias[layer_index][k] += self.learning_rate * ( bias_changes[
610             layer_index][k] / float(self.batch_size) )
611
612 def plot_models(loss_SGD,loss_SGDPlus,loss_SGDAdam,title):
613     plt.figure(figsize=(6,4))
614     plt.plot(loss_SGD, label='SGD')

```

```

592     plt.plot(loss_SGDPlus, label='SGD+')
593     plt.plot(loss_SGDAdam, label='Adam')
594
595     plt.title(title)
596     plt.xlabel('Iteration')
597     plt.ylabel('Loss')
598     plt.legend()
599     plt.savefig(f'{title}.png', bbox_inches='tight')
600     return
601
602 def performance_analysis(lr,type):
603     if type == '1':
604         title = f'One_Neuron_Models_Loss_lr={lr}'
605         cgp_SGD = OneNeuronSGD(
606             one_neuron_model = True,
607             expressions = ['xw=ab*xa+bc*xb+cd*xc+ac*xd'],
608             output_vars = ['xw'],
609             dataset_size = 5000,
610             learning_rate = lr,
611             training_iterations = 40000,
612             batch_size = 8,
613             display_loss_how_often = 100,
614             debug = True,
615         )
616
617         cgp_SGDPlus = OneNeuronSGDPlus(
618             beta=0.95,
619             one_neuron_model = True,
620             expressions = ['xw=ab*xa+bc*xb+cd*xc+ac*xd'],
621             output_vars = ['xw'],
622             dataset_size = 5000,
623             learning_rate = lr,
624             training_iterations = 40000,
625             batch_size = 8,
626             display_loss_how_often = 100,
627             debug = True,
628         )
629
630         cgp_SGDAdam = OneNeuronSGDAdam(
631             beta_1 = 0.95,
632             beta_2 = 0.9,
633             epsilon = 1e-8,
634             one_neuron_model = True,
635             expressions = ['xw=ab*xa+bc*xb+cd*xc+ac*xd'],
636             output_vars = ['xw'],
637             dataset_size = 5000,
638             learning_rate = lr,
639             training_iterations = 40000,
640             batch_size = 8,
641             display_loss_how_often = 100,
642             debug = True,
643         )
644
645         cgp_SGD.parse_expressions()
646         cgp_SGDPlus.parse_expressions()
647         cgp_SGDAdam.parse_expressions()
648
649         training_data = cgp_SGD.gen_training_data()
650         results = []
651
652         start_time = time.time()
653         loss_SGD = cgp_SGD.run_training_loop_one_neuron_model( training_data )
654         end_time = time.time()
655         results.append(['SGD', end_time-start_time, loss_SGD[-1], np.min(loss_SGD[10:])
656             ])
657
658         start_time = time.time()
659         loss_SGDPlus = cgp_SGDPlus.run_training_loop_one_neuron_model( training_data )
660         end_time = time.time()
661         results.append(['SGD+', end_time-start_time, loss_SGDPlus[-1], np.min(
662             loss_SGDPlus[10:])])
663
664         start_time = time.time()

```

```

663     loss_SGDAdam = cgp_SGDAdam.run_training_loop_one_neuron_model( training_data )
664     end_time = time.time()
665     results.append(['Adam', end_time-start_time, loss_SGDAdam[-1], np.min(
        loss_SGDAdam[10:]))
666
667 elif type == '2':
668     title = f'Multi_Neuron_Models_Loss_lr={lr}'
669
670     cgp_SGD = MultiNeuronSGD(
671         num_layers = 3,
672         layers_config = [4,2,1],
673         expressions = ['xw=ap*xp+aq*xq+ar*xr+as*xs',
674                       'xz=bp*xp+bq*xq+br*xr+bs*xs',
675                       'xo=cp*xw+cq*xz'],
676         output_vars = ['xo'],
677         dataset_size = 5000,
678         learning_rate = lr,
679         training_iterations = 20000,
680         batch_size = 8,
681         display_loss_how_often = 100,
682         debug = True,
683     )
684     cgp_SGD.parse_multi_layer_expressions()
685
686     cgp_SGDPlus = MultiNeuronSGDPlus(
687         beta = 0.95,
688         num_layers = 3,
689         layers_config = [4,2,1],
690         expressions = ['xw=ap*xp+aq*xq+ar*xr+as*xs',
691                       'xz=bp*xp+bq*xq+br*xr+bs*xs',
692                       'xo=cp*xw+cq*xz'],
693         output_vars = ['xo'],
694         dataset_size = 5000,
695         learning_rate = lr,
696         training_iterations = 20000,
697         batch_size = 8,
698         display_loss_how_often = 100,
699         debug = True,
700     )
701     cgp_SGDPlus.parse_multi_layer_expressions()
702
703     cgp_SGDAdam = MultiNeuronSGDAdam(
704         beta_1 = 0.8,
705         beta_2 = 0.9,
706         epsilon = 1e-8,
707         num_layers = 3,
708         layers_config = [4,2,1],
709         expressions = ['xw=ap*xp+aq*xq+ar*xr+as*xs',
710                       'xz=bp*xp+bq*xq+br*xr+bs*xs',
711                       'xo=cp*xw+cq*xz'],
712         output_vars = ['xo'],
713         dataset_size = 5000,
714         learning_rate = lr,
715         training_iterations = 20000,
716         batch_size = 8,
717         display_loss_how_often = 100,
718         debug = True,
719     )
720     cgp_SGDAdam.parse_multi_layer_expressions()
721
722     training_data = cgp_SGD.gen_training_data()
723     results = []
724
725     start_time = time.time()
726     loss_SGD = cgp_SGD.run_training_loop_multi_neuron_model( training_data )
727     end_time = time.time()
728     results.append(['SGD', end_time-start_time, loss_SGD[-1], np.min(loss_SGD[10:])
        ])
729
730     start_time = time.time()
731     loss_SGDPlus = cgp_SGDPlus.run_training_loop_multi_neuron_model( training_data )
732     end_time = time.time()

```

```

733     results.append(['SGD+', end_time-start_time, loss_SGDPlus[-1], np.min(
734         loss_SGDPlus[10:]))
735
736     start_time = time.time()
737     loss_SGDAdam = cgp_SGDAdam.run_training_loop_multi_neuron_model( training_data )
738     end_time = time.time()
739     results.append(['Adam', end_time-start_time, loss_SGDAdam[-1], np.min(
740         loss_SGDAdam[10:]))
741
742     headers = ['Optimizer', 'Time_Taken', 'Final_Loss', 'Minimum_Loss']
743     print(tabulate(results, headers=headers, tablefmt='latex'))
744     plot_models(loss_SGD,loss_SGDPlus,loss_SGDAdam, title)
745
746 def AdamHyperparameters(type):
747     b1_list = [0.8, 0.95, 0.99]
748     b2_list = [0.89, 0.9, 0.95]
749     results = []
750     for b1 in b1_list:
751         for b2 in b2_list:
752             if type == '1':
753                 cgp_SGDAdam = OneNeuronSGDAdam(
754                     beta_1 = b1,
755                     beta_2 = b2,
756                     epsilon = 1e-8,
757                     one_neuron_model = True,
758                     expressions = ['xw=ab*xa+bc*xb+cd*xc+ac*xd'],
759                     output_vars = ['xw'],
760                     dataset_size = 5000,
761                     learning_rate = 1e-3,
762                     training_iterations = 40000,
763                     batch_size = 8,
764                     display_loss_how_often = 100,
765                     debug = True,
766                 )
767                 cgp_SGDAdam.parse_expressions()
768                 training_data = cgp_SGDAdam.gen_training_data()
769                 start_time = time.time()
770                 loss_SGDAdam = cgp_SGDAdam.run_training_loop_one_neuron_model(
771                     training_data)
772                 end_time = time.time()
773             elif type == '2':
774                 cgp_SGDAdam = MultiNeuronSGDAdam(
775                     beta_1 = b1,
776                     beta_2 = b2,
777                     epsilon = 1e-8,
778                     num_layers = 3,
779                     layers_config = [4,2,1],
780                     expressions = ['xw=ap*xp+aq*xq+ar*xr+as*xs',
781                                   'xz=bp*xp+bq*xq+br*xr+bs*xs',
782                                   'xo=cp*xw+cq*xz'],
783                     output_vars = ['xo'],
784                     dataset_size = 5000,
785                     learning_rate = 9e-2,
786                     training_iterations = 20000,
787                     batch_size = 8,
788                     display_loss_how_often = 100,
789                     debug = True,
790                 )
791                 cgp_SGDAdam.parse_multi_layer_expressions()
792                 training_data = cgp_SGDAdam.gen_training_data()
793
794                 start_time = time.time()
795                 loss_SGDAdam = cgp_SGDAdam.run_training_loop_multi_neuron_model(
796                     training_data)
797                 end_time = time.time()
798                 results.append([b1, b2, end_time-start_time, loss_SGDAdam[-1], np.min(
799                     loss_SGDAdam[10:]))
800
801     headers = ['beta_1', 'beta_2', 'Time_Taken', 'Final_Loss', 'Minimum_Loss']
802     print(tabulate(results, headers=headers, tablefmt='latex'))
803
804 if __name__ == '__main__':
805     if sys.argv[2] == 'Performance_Analysis':

```

```

801         for lr in [1e-4]:# [1e-2, 1e-3, 1e-4]:
802             performance_analysis(lr,sys.argv[1])
803     elif sys.argv[2] == 'Hyperparameters':
804         AdamHyperparameters(sys.argv[1])

```

5 Source Code Extra Credit

```

1  import numpy as np
2  from ComputationalGraphPrimer import *
3  from main import OneNeuronSGD, MultiNeuronSGD
4
5  class OneNeuronSGDNoNormalization(ComputationalGraphPrimer):
6      def __init__(self, *args, ** kwargs):
7          super().__init__(*args, ** kwargs) # Inherit from CGP class
8
9      def run_training_loop_one_neuron_model(self, training_data):
10         self.vals_for_learnable_params = {param: random.uniform(0,1) for param in self.
            learnable_params}
11         self.bias = random.uniform(0,1)
12
13         class DataLoader:
14             def __init__(self, training_data, batch_size):
15                 self.training_data = training_data
16                 self.batch_size = batch_size
17                 self.class_0_samples = [(item, 0) for item in self.training_data[0]]
18                 self.class_1_samples = [(item, 1) for item in self.training_data[1]]
19
20             def __len__(self):
21                 return len(self.training_data[0]) + len(self.training_data[1])
22
23             def _getitem(self):
24                 cointoss = random.choice([0,1])
25                 if cointoss == 0:
26                     return random.choice(self.class_0_samples)
27                 else:
28                     return random.choice(self.class_1_samples)
29
30             def getbatch(self):
31                 batch_data, batch_labels = [], []
32                 for _ in range(self.batch_size):
33                     item = self._getitem()
34                     batch_data.append(item[0])
35                     batch_labels.append(item[1])
36                 batch_data = [item for item in batch_data]
37                 batch = [batch_data, batch_labels]
38                 return batch
39
40         data_loader = DataLoader(training_data, batch_size=self.batch_size)
41         loss_running_record = []
42         i = 0
43         avg_loss_over_iterations = 0.0
44         for i in range(self.training_iterations):
45             data = data_loader.getbatch()
46             data_tuples_in_batch = data[0]
47             class_labels_in_batch = data[1]
48             y_preds, deriv_sigmoids = self.forward_prop_one_neuron_model(
                data_tuples_in_batch)
49             loss = sum([(abs(class_labels_in_batch[i] - y_preds[i]))**2 for i in range(
                len(class_labels_in_batch))]) ## Find loss
50             avg_loss_over_iterations += loss / float(len(class_labels_in_batch))
51             if i%(self.display_loss_how_often) == 0:
52                 avg_loss_over_iterations /= self.display_loss_how_often
53                 loss_running_record.append(avg_loss_over_iterations)
54                 print("[iter=%d] loss=%0.4f" % (i+1, avg_loss_over_iterations))
55                 avg_loss_over_iterations = 0.0
56             y_errors_in_batch = list(map(operator.sub, class_labels_in_batch, y_preds))
57             self.backprop_and_update_params_one_neuron_model(data_tuples_in_batch,
                y_preds, y_errors_in_batch, deriv_sigmoids) ## BACKPROP loss
58         return loss_running_record
59

```

```

60 class MultiNeuronSGDNoNormalization(ComputationalGraphPrimer):
61     def __init__(self, *args, ** kwargs):
62         super().__init__(*args, ** kwargs) # Inherit from CGP class
63
64     def run_training_loop_multi_neuron_model(self, training_data):
65
66         class DataLoader:
67             def __init__(self, training_data, batch_size):
68                 self.training_data = training_data
69                 self.batch_size = batch_size
70                 self.class_0_samples = [(item, 0) for item in self.training_data[0]]
71                 self.class_1_samples = [(item, 1) for item in self.training_data[1]]
72                 ## Associate label 0 with each sample
73                 ## Associate label 1 with each sample
74
75             def __len__(self):
76                 return len(self.training_data[0]) + len(self.training_data[1])
77
78             def _getitem(self):
79                 cointoss = random.choice([0,1])
80                 batch is created by getbatch(), we want the
81
82                 ## When a
83                 ## samples
84                 ## to be
85                 ## chosen
86                 ## randomly
87                 ## from the
88                 ## two lists
89
90                 if cointoss == 0:
91                     return random.choice(self.class_0_samples)
92                 else:
93                     return random.choice(self.class_1_samples)
94
95             def getbatch(self):
96                 batch_data, batch_labels = [], []
97                 for _ in range(self.batch_size):
98                     item = self._getitem()
99                     batch_data.append(item[0])
100                     batch_labels.append(item[1])
101                 batch_data = [item for item in batch_data]
102                 batch = [batch_data, batch_labels]
103                 return batch
104
105         self.vals_for_learnable_params = {param: random.uniform(0,1) for param in self.
106             learnable_params}
107         self.bias = {i : [random.uniform(0,1) for j in range( self.layers_config[i] )
108             ] for i in range(1, self.num_layers)}
109         data_loader = DataLoader(training_data, batch_size=self.batch_size)
110         loss_running_record = []
111         i = 0
112         avg_loss_over_iterations = 0.0
113         for i in range(self.training_iterations):
114             data = data_loader.getbatch()
115             data_tuples = data[0]
116             class_labels = data[1]
117             self.forward_prop_multi_neuron_model(data_tuples)
118             ## FORW PROP works by side-effect
119             predicted_labels_for_batch = self.forw_prop_vals_at_layers[self.num_layers
120                 -1]
121             ## Predictions from FORW PROP
122             y_preds = [item for sublist in predicted_labels_for_batch for item in
123                 sublist]
124             ## Get numeric vals for predictions
125             loss = sum([(abs(class_labels[i] - y_preds[i]))**2 for i in range(len(
126                 class_labels))])
127             ## Calculate loss for batch
128             loss_avg = loss / float(len(class_labels))
129             ## Average the loss over
130             batch
131             avg_loss_over_iterations += loss_avg
132             ## Add to Average
133             loss over iterations
134         if i%(self.display_loss_how_often) == 0:
135             avg_loss_over_iterations /= self.display_loss_how_often
136             loss_running_record.append(avg_loss_over_iterations)
137             print("[iter=%d] loss = %.4f" % (i+1, avg_loss_over_iterations))
138             ## Display avg loss

```

```

114         avg_loss_over_iterations = 0.0
115         initialize avg-over-iterations loss
116         y_errors_in_batch = list(map(operator.sub, class_labels, y_preds))
117         self.backprop_and_update_params_multi_neuron_model(y_preds,
118             y_errors_in_batch)
119         return loss_running_record
120
121     class OneNeuronSGDTruncated(OneNeuronSGD):
122         def __init__(self, *args, ** kwargs):
123             super().__init__(*args, ** kwargs)
124
125         class AutogradCustomization(torch.nn.Module):
126             def __init__(self, cgp, num_samples_per_class):
127                 super(ComputationalGraphPrimer.AutogradCustomization, self).__init__()
128                 self.cgp = cgp
129                 self.num_samples_per_class = num_samples_per_class
130
131             def gen_training_data(self):
132                 mean1,mean2 = [3.0,3.0], [5.0,5.0]
133                 covar1,covar2 = [[1.0,0.0], [0.0,1.0]], [[1.0,0.0], [0.0,1.0]]
134                 inv_covar1 = np.linalg.inv(covar1)
135                 inv_covar2 = np.linalg.inv(covar2)
136
137                 def mahalanobis_dist(x, mean, inv_covar):
138                     return np.sqrt((x - mean).T @ inv_covar @ (x - mean))
139
140                 data1 = []
141                 data2 = []
142                 for x in np.random.multivariate_normal(mean1, covar1, self.
143                     num_samples_per_class):
144                     if mahalanobis_dist(x, mean1, inv_covar1) <= 5:
145                         data1.append((list(x),1))
146                 for x in np.random.multivariate_normal(mean2, covar2, self.
147                     num_samples_per_class):
148                     if mahalanobis_dist(x, mean2, inv_covar2) <= 5:
149                         data2.append((list(x),2))
150
151                 training_data = data1 + data2
152                 random.shuffle( training_data )
153                 self.training_data = training_data
154
155     class MultiNeuronSGDTruncated(MultiNeuronSGD):
156         def __init__(self, *args, ** kwargs):
157             super().__init__(*args, ** kwargs)
158
159         class AutogradCustomization(torch.nn.Module):
160             def __init__(self, cgp, num_samples_per_class):
161                 super(ComputationalGraphPrimer.AutogradCustomization, self).__init__()
162                 self.cgp = cgp
163                 self.num_samples_per_class = num_samples_per_class
164
165             def gen_training_data(self):
166                 mean1,mean2 = [3.0,3.0], [5.0,5.0]
167                 covar1,covar2 = [[1.0,0.0], [0.0,1.0]], [[1.0,0.0], [0.0,1.0]]
168                 inv_covar1 = np.linalg.inv(covar1)
169                 inv_covar2 = np.linalg.inv(covar2)
170
171                 def mahalanobis_dist(x, mean, inv_covar):
172                     return np.sqrt((x - mean).T @ inv_covar @ (x - mean))
173
174                 data1 = []
175                 data2 = []
176                 for x in np.random.multivariate_normal(mean1, covar1, self.
177                     num_samples_per_class):
178                     if mahalanobis_dist(x, mean1, inv_covar1) <= 5:
179                         data1.append((list(x),1))
180                 for x in np.random.multivariate_normal(mean2, covar2, self.
181                     num_samples_per_class):
182                     if mahalanobis_dist(x, mean2, inv_covar2) <= 5:
183                         data2.append((list(x),2))
184
185                 training_data = data1 + data2

```

```

180         random.shuffle( training_data )
181         self.training_data = training_data
182
183     def plot_comparisons(loss1,loss2,name1,name2,title):
184         plt.figure(figsize=(6,4))
185         plt.plot(loss1, label=name1)
186         plt.plot(loss2, label=name2)
187
188         plt.title(title)
189         plt.xlabel('Iteration')
190         plt.ylabel('Loss')
191         plt.legend()
192         plt.savefig(f'{title}.png', bbox_inches='tight')
193         return
194
195     def normalization_comparisons():
196         ##### One-Neuron #####
197         cgp = OneNeuronSGD(
198             one_neuron_model = True,
199             expressions = ['xw=ab*xa+bc*xb+cd*xc+ac*xd'],
200             output_vars = ['xw'],
201             dataset_size = 5000,
202             learning_rate = 1e-3,
203             training_iterations = 40000,
204             batch_size = 8,
205             display_loss_how_often = 100,
206             debug = True,
207         )
208         cgp.parse_expressions()
209         training_data = cgp.gen_training_data()
210         loss_with_normalization = cgp.run_training_loop_one_neuron_model( training_data )
211
212         cgp = OneNeuronSGDNoNormalization(
213             one_neuron_model = True,
214             expressions = ['xw=ab*xa+bc*xb+cd*xc+ac*xd'],
215             output_vars = ['xw'],
216             dataset_size = 5000,
217             learning_rate = 1e-3,
218             training_iterations = 40000,
219             batch_size = 8,
220             display_loss_how_often = 100,
221             debug = True,
222         )
223         cgp.parse_expressions()
224         training_data = cgp.gen_training_data()
225         loss_without_normalization = cgp.run_training_loop_one_neuron_model( training_data )
226         plot_comparisons(loss_with_normalization, loss_without_normalization, 'With
Normalization', 'Without Normalization', 'OneNeuronModelWithandWithout
Normalization')
227
228         ##### Multi-Neuron #####
229         cgp = MultiNeuronSGD(
230             num_layers = 3,
231             layers_config = [4,2,1], # num of nodes in each
layer
232             expressions = ['xw=ap*xp+aq*xq+ar*xr+as*xs',
233                           'xz=bp*xp+bq*xq+br*xr+bs*xs',
234                           'xo=cp*xw+cq*xz'],
235             output_vars = ['xo'],
236             dataset_size = 5000,
237             learning_rate = 9e-2,
238             training_iterations = 20000,
239             batch_size = 8,
240             display_loss_how_often = 100,
241             debug = True,
242         )
243         cgp.parse_multi_layer_expressions()
244         training_data = cgp.gen_training_data()
245         loss_with_normalization = cgp.run_training_loop_multi_neuron_model( training_data )
246
247         cgp = MultiNeuronSGDNoNormalization(
248             num_layers = 3,

```

```

249         layers_config = [4,2,1],                                # num of nodes in each
250         layer
251     expressions = ['xw=ap*xp+aq*xq+ar*xr+as*xs',
252                   'xz=bp*xp+bq*xq+br*xr+bs*xs',
253                   'xo=cp*xw+cq*xz'],
254     output_vars = ['xo'],
255     dataset_size = 5000,
256     learning_rate = 9e-2,
257     training_iterations = 20000,
258     batch_size = 8,
259     display_loss_how_often = 100,
260     debug = True,
261 )
262 cgp.parse_multi_layer_expressions()
263 training_data = cgp.gen_training_data()
264 loss_without_normalization = cgp.run_training_loop_multi_neuron_model( training_data
265 )
266 plot_comparisons(loss_with_normalization, loss_without_normalization, 'With_
267 Normalization', 'Without_Normalization', 'Multi_Neuron_Model_With_and_Without_
268 Normalization')
269 return
270
271 def truncation_comparison():
272     ##### One-Neuron #####
273     cgp = OneNeuronSGD(
274         one_neuron_model = True,
275         expressions = ['xw=ab*xa+bc*xb+cd*xc+ac*xd'],
276         output_vars = ['xw'],
277         dataset_size = 5000,
278         learning_rate = 1e-3,
279         training_iterations = 40000,
280         batch_size = 8,
281         display_loss_how_often = 100,
282         debug = True,
283     )
284     cgp.parse_expressions()
285     training_data = cgp.gen_training_data()
286     loss_without_truncation = cgp.run_training_loop_one_neuron_model( training_data )
287
288     cgp = OneNeuronSGDTruncated(
289         one_neuron_model = True,
290         expressions = ['xw=ab*xa+bc*xb+cd*xc+ac*xd'],
291         output_vars = ['xw'],
292         dataset_size = 5000,
293         learning_rate = 1e-3,
294         training_iterations = 40000,
295         batch_size = 8,
296         display_loss_how_often = 100,
297         debug = True,
298     )
299     cgp.parse_expressions()
300     training_data = cgp.gen_training_data()
301     loss_with_truncation = cgp.run_training_loop_one_neuron_model( training_data )
302     plot_comparisons(loss_without_truncation, loss_with_truncation, 'Normalized_Without_
303 Truncation', 'Normalized_With_Truncation', 'One_Neuron_Model_With_and_Without_
304 Truncation')
305
306     ##### Multi-Neuron #####
307     cgp = MultiNeuronSGD(
308         num_layers = 3,
309         layers_config = [4,2,1],                                # num of nodes in each
310         layer
311     expressions = ['xw=ap*xp+aq*xq+ar*xr+as*xs',
312                   'xz=bp*xp+bq*xq+br*xr+bs*xs',
313                   'xo=cp*xw+cq*xz'],
314     output_vars = ['xo'],
315     dataset_size = 5000,
316     learning_rate = 9e-2,
317     training_iterations = 20000,
318     batch_size = 8,
319     display_loss_how_often = 100,
320     debug = True,
321 )

```

```

315     cgp.parse_multi_layer_expressions()
316     training_data = cgp.gen_training_data()
317     loss_without_truncation = cgp.run_training_loop_multi_neuron_model( training_data )
318
319     cgp = MultiNeuronSGDTruncated(
320         num_layers = 3,
321         layers_config = [4,2,1],                                # num of nodes in each
322                             layer
323         expressions = ['xw=ap*xp+aq*xq+ar*xr+as*xs',
324                             'xz=bp*xp+bq*xq+br*xr+bs*xs',
325                             'xo=cp*xw+cq*xz'],
326         output_vars = ['xo'],
327         dataset_size = 5000,
328         learning_rate = 9e-2,
329         training_iterations = 20000,
330         batch_size = 8,
331         display_loss_how_often = 100,
332         debug = True,
333     )
334     cgp.parse_multi_layer_expressions()
335     training_data = cgp.gen_training_data()
336     loss_with_truncation = cgp.run_training_loop_multi_neuron_model( training_data )
337     plot_comparisons(loss_without_truncation, loss_with_truncation, 'Normalized Without Truncation', 'Normalized With Truncation', 'Multi Neuron Model With and Without Truncation')
338
339 if __name__ == '__main__':
340     normalization_comparisons()
341     truncation_comparison()

```
